COS 331- Operating Systems II (2 units)

Course Contents

Concurrency: States and state diagrams, structures, Dispatching and context switching; interrupts, concurrent execution, mutual exclusion problem and some solutions. Deadlock; Models and mechanisms (semaphores, monitors, etc), producer-consumer problem and synchronization; Multiprocessor issues; scheduling and dispatching; Memory management; Overlays, swapping and partition, paging and segmentation, placement and replacement policies, working sets and trashing, caching.

1.0 INTRODUCTION

Concurrency

Permit us to throw some light on the following two terms: Concurrency and parallelism that are associated with computer jobs execution.

Concurrency means executing multiple tasks at the same time but not necessarily simultaneously. The term concurrency is usually achieved in a uniprocessor system. In a single-core environment, concurrency is achieved via a process called **context-switching**. If it's a multi-core environment, concurrency can be achieved through parallelism.

Parallelism is concerned with utilizing multiple processors/cores to perform two or more tasks simultaneously. Parallel computing in computer science refers to the process of performing multiple calculations simultaneously. This can only be achieved in a multiprocessor system.

- ***In a single-core environment, concurrency happens*** with tasks executing over the same time period via context switching. i.e., at a particular time period, only a single task gets executed.
- ***In a multi-core environment, concurrency can be achieved via parallelism*** in which multiple tasks are executed simultaneously.

Parallel processing or parallelism is being widely used in computer architectures ranging from supercomputers to workstations, because of its promise of high performance at a relatively low cost.

Parallel processing in multiprocessor computer is said to be true parallel processing and parallel processing in uniprocessor computer is said to be simulated or virtual parallel processing. The following figures demonstrate the difference between true and virtual parallel processing by the following figures (see figure 1):

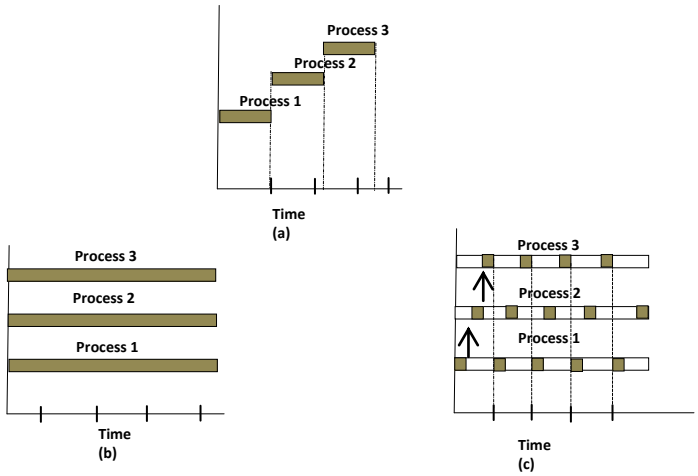


Figure 1 (a) Serial processing (b) True parallel processing with multiple processor
(c) Parallel processing simulated by switching

Figure 1 (a) represents the serial processing means next processing is started when the previous process must be completed. In figure 1(b) all three process are running in one clock cycle of three processors. In figure1 (c) all three process are also running in one clock cycle but each process are getting only 1/3 of actual clock cycle on each clock cycle and the CPU is switching from on process to other in its clock cycle. When one process is running all other process must wait for their turn. If we see in figure1(b) then we will find that at

any particular point only one process is running and other are waiting. But for one clock time all three process are running. So in uniprocessor system the parallel processing is called virtual parallel processing.

1. Process States and state diagrams

All modern computers often do several things at the same time. People work with personal computers but may not be fully aware of this fact, so a few examples may make the point clearer. First consider a Web server. Requests come in from all over asking for Web pages. When a request comes in, the server checks to see if the page needed is in the cache. Imagine, the memory hierarchy! If the page is in the cache, it is sent back; if it is not, a disk request is started to fetch it.

However, from the CPU's perspective, disk requests sometimes take long time to complete. While waiting for the disk request to complete, many more requests may come in. If there are multiple disks present, some or all of them may be fired off to other disks long before the first request is satisfied. Clearly some way is needed to model and control this concurrency. Processes (and especially threads) can help here. Now consider a user PC. When the system is booted, many processes are secretly started, often unknown to the user. For example, a process may be started up to wait for incoming e-mail. Another process may run on behalf of the antivirus program to check periodically if any new virus definitions are available. In addition, explicit user processes may be running, printing files and burning a CDROM, all while the user is surfing the Web. All this activity has to be managed, and a multiprogramming system supporting multiple processes comes in very handy here.

Process concepts

A question that arises in discussing operating systems involves what to call all the CPU activities. A batch system executes jobs, whereas a time-shared system has user's programs, or tasks. Even on a single-user system such as Microsoft Windows, a user may be able to run several programs at one time. In many respects, all these activities are similar, so we call all of them processes.

- A process is a program in execution. The execution of a process must progress in a sequential fashion.
- Process is a dynamic entity i.e., a program in execution.
- Processes exist in a limited span of time.
- A process has object program, data, resources and the status of the process on execution.
- A process is not a closed system; there is a communication between processes through a shared environment.

What is a Process?

A process is defined as an entity which represents the basic unit of work to be implemented in the system.
Components of a process are:

I. Object Program: - Code to be executed
II. Data: - Data to be used for executing the program
III. Resources:- While executing the program, it may require some resources
IV. Status:- verifies the status of the process execution.

A process can run to completion only when all requested resources have been allocated to the process. Two or more processes could be executing the same program, each using their own data and resources.
A program by itself is not a process. It is a static entity made up of program statement while process is a dynamic entity. Program contains the instructions to be executed by processor.
A program takes a space at single place in main memory and continues to stay there. A program does not perform any action by itself.

Programs versus Process

Program	Process
Static entity	Dynamic entity
Exist in a single space and continue to exist	Exist in a limited span of time.
Programs contain instructions	Executes instruction

There are two types of processes:
Sequential Processes: Execution progresses in a sequential fashion, i.e., one after the other. So, at any point in time, at most one process is being executed.
Concurrent Processes: There are two types of concurrent processes.
(i) **True Concurrency (Multiprocessing):** Two or more processes are executed simultaneously in a multiprocessor environment. So, supports real parallelism.
(ii) **Apparent Concurrency (Multiprogramming):** Two or more processes are executed in parallel in a uniprocessor environment.

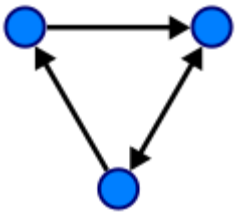
State Diagram

A **state diagram** is used in computer science and related fields to describe the behavior of systems. State diagrams require that the system is composed of a finite number of states. Sometimes, this is indeed the case, while at other times this is a reasonable abstraction. Many forms of state diagrams exist, which differ slightly and have different semantics.

State diagrams provide an abstract description of a system's behavior. This behavior is analyzed and represented by a series of events that can occur in one or more possible states. Hereby "each diagram usually represents objects of a single class and track the different states of its objects through the system".

State diagrams can be used to graphically represent finite-state machines (also called finite automata). This was introduced by Claude Shannon and Warren Weaver in their 1949 book *The Mathematical Theory of Communication*.

Directed graph]



A directed graph

A classic form of state diagram for a finite automaton (FA) is a directed graph with the following elements $(Q, \Sigma, Z, \delta, q_0, F)$:

- **Vertices Q**: a finite set of states, normally represented by circles and labeled with unique designator symbols or words written inside them
- **Input symbols Σ**: a finite collection of input symbols or designators
- **Output symbols Z**: a finite collection of output symbols or designators

The output function ω represents the mapping of ordered pairs of input symbols and states onto output symbols, denoted mathematically as $\boldsymbol{\omega} : \boldsymbol{\Sigma} \times \boldsymbol{Q} \rightarrow \boldsymbol{Z}$.

- **Edges δ**: represent transitions from one state to another as caused by the input (identified by their symbols drawn on the edges). An edge is usually drawn as an arrow directed from the present state to the next state. This mapping describes the state transition caused by an input. This is written mathematically as $\boldsymbol{\delta} : \boldsymbol{Q} \times \boldsymbol{\Sigma} \rightarrow \boldsymbol{Q}$, so δ (the transition function) in the definition of the FA is given by both the pair of vertices connected by an edge and the symbol on an edge in a diagram representing this FA. Item **δ(q, a) = p** in the definition of the FA means that from the state named **q** under input symbol **a**,

the transition to the state **p** occurs in this machine. In the diagram representing this FA, this is represented by an edge labeled by **a** pointing from the vertex labeled by **q** to the vertex labeled by **p**.

- **Start state q_0**: (not shown in the examples below). The start state $q_0 \in Q$ is usually represented by an arrow with no origin pointing to the state. In older texts, the start state is not shown and must be inferred from the text.
- **Accepting state(s) F**: If used, for example for accepting automata, $F \in Q$ is the accepting state. It is usually drawn as a double circle. Sometimes the accept state(s) function as "**F**inal" (halt, trapped) states.

State Diagram and Operating System

The concept of program

From the programmers perspective a program is an *ordered set of instructions*. On the other hand, from the point of view of an operating system, it is an *executable file* stored in the secondary (auxiliary) memory, typically on a disk. The file is normally stored in the secondary memory and can be accessed through the corresponding directory entry and file description.

The concept of process

In operating system terminology, the notion of **process** is used relating to execution instead of the term program. It designates a commission, or quantum of work dealt with as an entity. Consequently, required resources like address space etc. will be allocated typically on a process basis. Lastly, to execute a process, a processor is to be allocated for. This is expressed in operating system terminology by saying that the process is to be scheduled for execution.

Each process has a *lifecycle,* which consists of creation, an execution phase and termination. In order to execute a program prior a corresponding process is to be created. A process will be brought into existence by using system services (system calls or supervisor macros). The creation of a process means commissioning the operating system to execute a program.

Process creation involves the following four main actions:

• setting up the process description,
• allocation of an address space and
• loading the program into the allocated address space and
• passing on the process description to the scheduler.

Process States in Operating System vs1

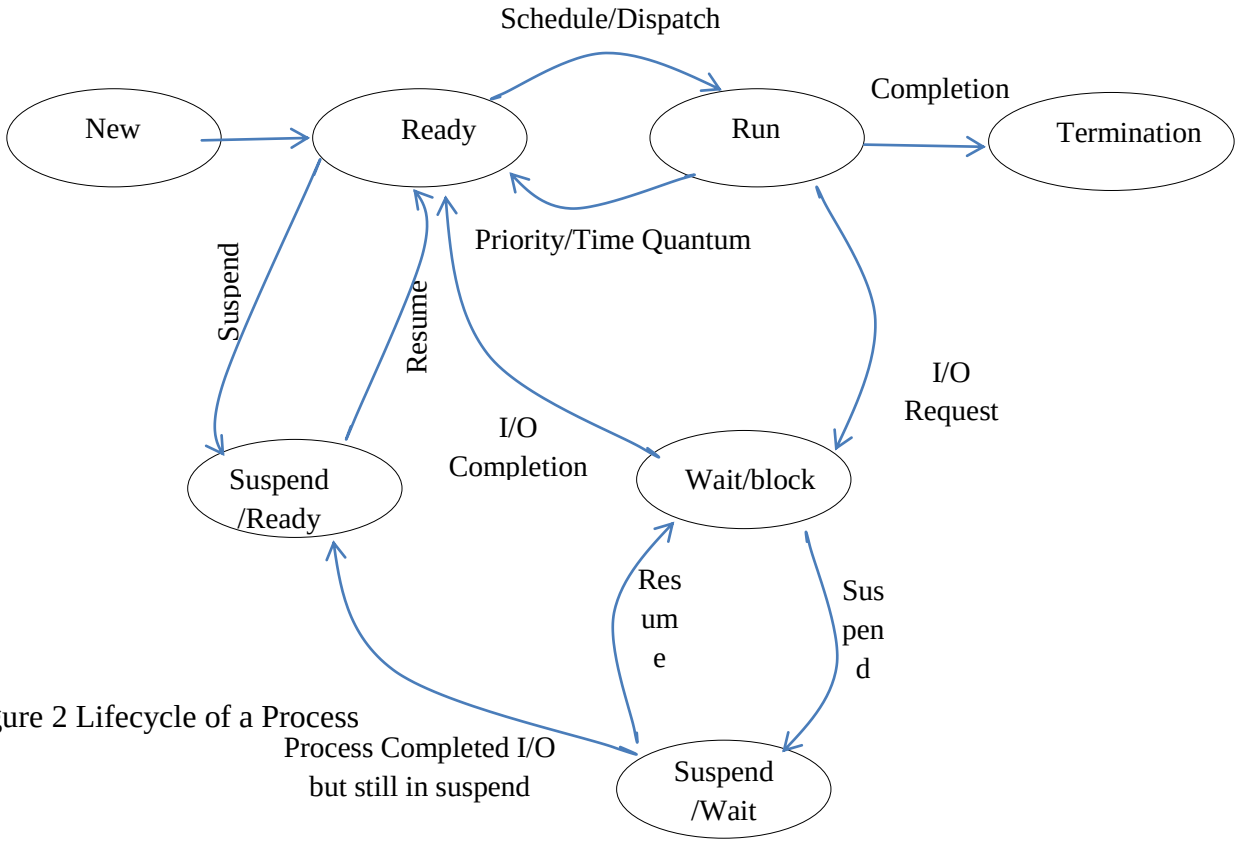
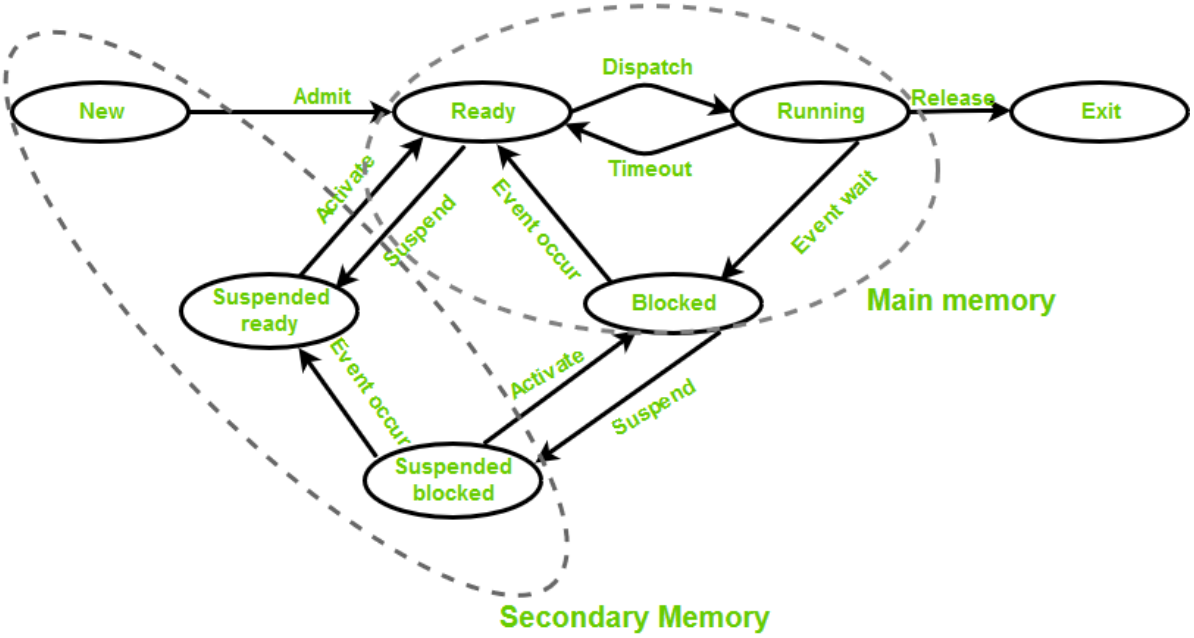


Figure 2 Lifecycle of a Process



The process, from its creation to completion, passes through various states. The minimum number of states is five. The names of the states are not standardized although the process may be in one of the following states during execution.

1. New

A program which is going to be picked up by the OS into the main memory is called a new process.

2. Ready

Whenever a process is created, it directly enters in the ready state, in which, it waits for the CPU to be assigned. The OS picks the new processes from the secondary memory and put all of them in the main memory.

The processes which are ready for the execution and reside in the main memory are called ready state processes. There can be many processes present in the ready state.

3. Running

One of the processes from the ready state will be chosen by the OS depending upon the scheduling algorithm. Hence, if we have only one CPU in our system, the number of running processes for a particular time will always be one. If we have n processors in the system then we can have n processes running simultaneously.

4. Block or waits

From the Running state, a process can make the transition to the block or wait state depending upon the scheduling algorithm or the intrinsic behavior of the process.

When a process waits for a certain resource to be assigned or for the input from the user then the OS move this process to the block or wait state and assigns the CPU to the other processes.

5. Completion or termination

When a process finishes its execution, it comes in the termination state. All the context of the process (Process Control Block) will also be deleted the process will be terminated by the Operating system.

6. Suspend ready

A process in the ready state, which is moved to secondary memory from the main memory due to lack of the resources (mainly primary memory) is called in the suspend ready state.

If the main memory is full and a higher priority process comes for the execution then the OS have to make the room for the process in the main memory by throwing the lower priority process out into the secondary

memory. The suspend ready processes remain in the secondary memory until the main memory gets available.

7. Suspend wait

Instead of removing the process from the ready queue, it's better to remove the blocked process which is waiting for some resources in the main memory. Since it is already waiting for some resource to get available hence it is better if it waits in the secondary memory and make room for the higher priority process. These processes complete their execution once the main memory gets available and their wait is finished.

Process States in Operating System vs2

The states of a process are as follows:

- **New (Create):** In this step, the process is about to be created but not yet created. It is the program that is present in secondary memory that will be picked up by OS to create the process.
- **Ready:** New -> Ready to run. After the creation of a process, the process enters the ready state i.e. the process is loaded into the main memory. The process here is ready to run and is waiting to get the CPU time for its execution. Processes that are ready for execution by the CPU are maintained in a queue called ready queue for ready processes.
- **Run:** The process is chosen from the ready queue by the CPU for execution and the instructions within the process are executed by any one of the available CPU cores.
- **Blocked or Wait:** Whenever the process requests access to I/O or needs input from the user or needs access to a critical region(the lock for which is already acquired) it enters the blocked or waits state. The process continues to wait in the main memory and does not require CPU. Once the I/O operation is completed the process goes to the ready state.
- **Terminated or Completed:** Process is killed as well as PCB is deleted. The resources allocated to the process will be released or deallocated.
- **Suspend Ready:** Process that was initially in the ready state but was swapped out of main memory(refer to Virtual Memory topic) and placed onto external storage by the scheduler is said to be in suspend ready state. The process will transition back to a ready state whenever the process is again brought onto the main memory.
- **Suspend wait or suspend blocked:** Similar to suspend ready but uses the process which was performing I/O operation and lack of main memory caused them to move to secondary memory. When work is finished it may go to suspend ready.

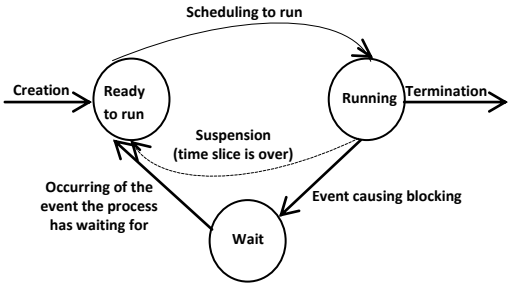
The execution phase of a process is under the control of the scheduler. It commences with creation of a process and lasts until the process is terminated. However, as far as scheduling is concerned, there are two basic models used by operating systems, termed as the *process model* and the *process-thread mode*. They differ essentially in the *granularity* **i.e., the size of the units by which work is allocated to processors or** the units of work which are scheduled as one entity.

In the **process model** the scheduling is performed on a *per process* basis, i.e. the smallest unit of work to be scheduled is a process. On the contrary, the **process-thread model** is a *finer grained* scheduling model, where smaller units of work, called *threads*, are scheduled as entities. In the following we will outline the process model, whereas the process-thread model will be briefly discussed in connection with the description of threads.

Process scheduling involves three key concepts:

i. the declaration of distinct **process states**,
ii. the specification of the **state transition diagram** and
iii. the statement of a **scheduling policy**.

As far as process states are concerned, there are *three basic states* connected with scheduling (see figure 3 below); the *ready to run* state, the *running* state as well as the *wait* (or *blocked*) state.



In the **ready to run state** processes are able to run when a processor is allocated for them. In this state they are waiting for the processor to be executed. In the **running state** they are in execution on the allocated processor. In the **wait state** they are suspended or blocked waiting for the occurrence of some events for getting again ready-to-run.

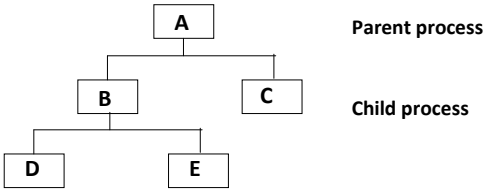


Figure 4. Process spawning

The Concept of Thread

The notion of thread was introduced in the framework of the process-thread model in order to express more parallelism in code than in the process model. This is achieved by declaration of smaller chunks of code, called *threads* (lightweight processes), within a process as an entity that can be executed concurrently or parallel. A **thread**, like a process, is a sequence of instructions. Threads are created *within and belonging to* processes. All the threads created within one process share the resources of the process, above all, the address space. Scheduling is also performed on a per thread basis. In other words, the process-thread model is a *finer grain scheduling model* than the process model.

Threads have a similar *lifecycle* as the processes and will be managed mainly in the same way as processes are. Initially each process will be created with one single thread. However, threads are usually allowed to create new ones using particular system calls. For instance, in OS/2 new threads can be created by issuing the DoscCreateThread system call. Then, typically for each process a thread tree will be created (Figure 5).

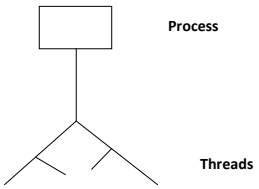
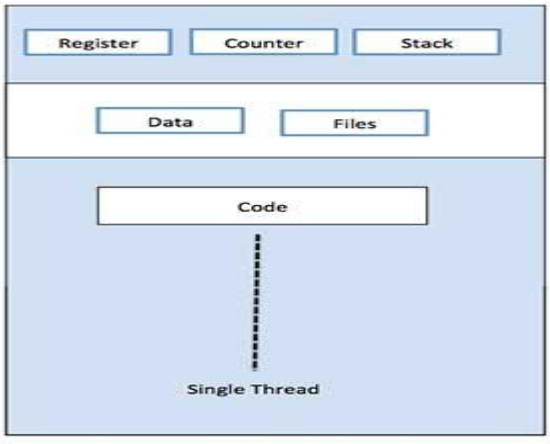


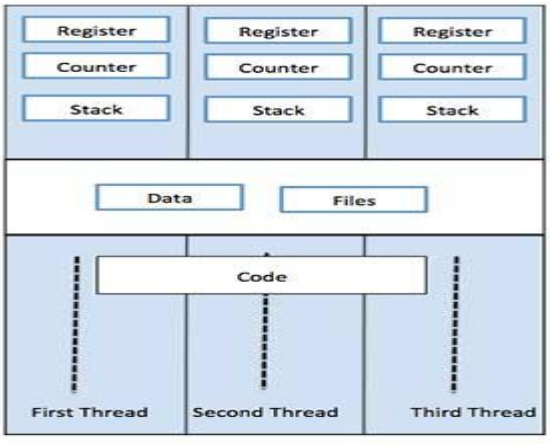
Figure 5. Thread tree

During creation each thread will be declared by a particular data structure mostly called *Thread Control Block* (*TCB*). The scheduling of threads will be performed in a similar way as described for processes.

Each thread belongs to exactly one process and no thread can exist outside a process. Each thread represents a separate flow of control. Threads have been successfully used in implementing network servers and web server. They also provide a suitable foundation for parallel execution of applications on shared memory multiprocessors. The following figure shows the working of a single-threaded and a multithreaded process.



Single Process P with single thread



Single Process P with three threads

Operations on the Process

1. Creation
Once the process is created, it will be ready and come into the ready queue (main memory) and will be ready for the execution.
2. Scheduling
Out of the many processes present in the ready queue, the Operating system chooses one process and start executing it. Selecting the process which is to be executed next, is known as scheduling.
3. Execution
Once the process is scheduled for the execution, the processor starts executing it. Process may come to the blocked or wait state during the execution then in that case the processor starts executing the other processes.
4. Deletion/killing
Once the purpose of the process gets over then the OS will kill the process. The Context of the process (PCB) will be deleted and the process gets terminated by the Operating system.

2.1 Structures (ready list, process control blocks, and so forth)
Ready list: The ready processes in the system are kept in a **queue** called the ready list. This list is sorted by the priority of the processes; the lowest priority process appears at the head of the list and the highest priority process is at the tail.

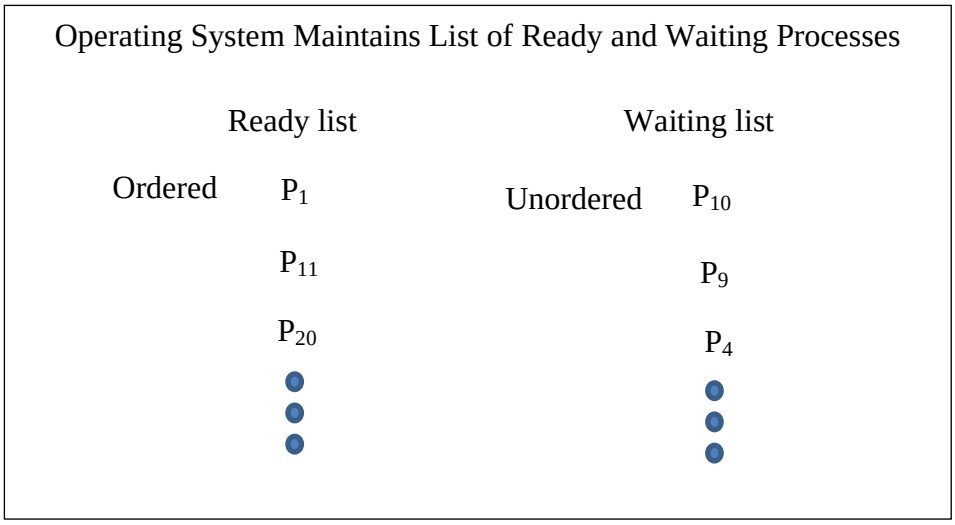


Fig. 2:Ready List and Waiting List

Process Control Block or PCB:

Usually, Operating systems *describe* a process by means of a description table call **Process Control Block or PCB**. PCB serves as a runtime description of the lifecycle of a process. Different operating systems are using, however, diverse designations for the process description table, like **Process Table** in UNIX, **Process Control Block** in VMS, **Process Information Block** in OS/2, or **Task Control Block** in IBM mainframe operating systems. A PCB contains all the information that can be relevant during the whole lifecycle of a process. It holds on the one hand, *basic data*, such as process identification number (PID), priority, process status (ready, running, dormant), hardware status (CPU registers, flags), scheduling information, I/O status (allocated devices, pending), Program counter, Scheduling priority, Credentials, A pointer to the process's parent process, Pointers to the process's child processes to locate the process's data, and instructions in memory to allocated resources, file management information (open file, close file, access right), description of the allocated address space, accounting information, etc.

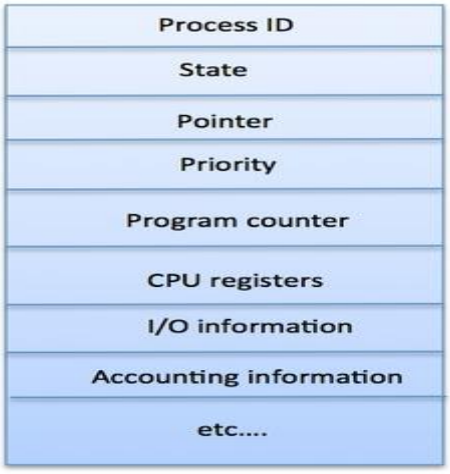
Implementation of Processes

Process Control Block (PCB)

A Process Control Block is a data structure maintained by the Operating System for every process. The PCB is identified by an integer process ID (PID). A PCB keeps all the information needed to keep track of a process as listed below in the table −

S.N.	Information & Description
1	**Process State** The current state of the process i.e., whether it is ready, running, waiting, or whatever.
2	**Process privileges** This is required to allow/disallow access to system resources.
3	**Process ID** Unique identification for each of the process in the operating system.
4	**Pointer** A pointer to parent process.
5	**Program Counter** Program Counter is a pointer to the address of the next instruction to be executed for this process.
6	**CPU registers** Various CPU registers where process need to be stored for execution for running state.
7	**CPU Scheduling Information** Process priority and other scheduling information which is required to schedule the process.
8	**Memory management information** This includes the information of page table, memory limits, Segment table depending on memory used by the operating system.
9	**Accounting information** This includes the amount of CPU used for process execution, time limits, execution ID etc.
10	**IO status information** This includes a list of I/O devices allocated to the process.

The architecture of a PCB is completely dependent on Operating System and may contain different information in different operating systems. Here is a simplified diagram of a PCB:



The PCB is maintained for a process throughout its lifetime, and is deleted once the process terminatesThese may include

- Process state
- Program counter
- Stack pointer
- Value of CPU registers when the process is suspended.
- CPU scheduling information such as priority
- The area of memory used by the process (memory management information)
- Accounting information: amount of CPU time used, time limits, process no, etc.
- I/O status information: list of I/O devices such as tape derives allocated for the process, a list of open files, etc.
- Pointer to the next process's PCB

Process control block includes CPU scheduling, I/O resource management, file management information etc. The PCB serves as the repository for any information which can vary from process to process.

2.2 Scheduling/Dispatching and Context switching

2.1 PROCESS SCHEDULING AND DISPATCHING

One of the most critical resources in any computer system is the central processor. Access to the central processor by any task occurs through the system scheduler/dispatcher modules.

Scheduling is the act of arranging a set of processes in some ordered way according to a well-defined strategy such as first come first serve (FCFS), last come first serve (LCFS), Priority, etc). The **SCHEDULER** is the module responsible for inserting processes into the ready-to- run queue and maintaining the structure of the queue. **Dispatching** is the operation of selecting a process from the ready-to- run queue and providing it with access to the central processor. The **DISPACHER** is the module that removes processes from the ready-to-run queue, establishes their machine state for execution, and transfers control to the process. In some systems, no distinction is made between the two modules in terms of functions and operations. However, in real-time system, these are distinguished. CPU can only be doing one thing at a time: if user process is executing, dispatcher is not: OS has lost control. How does OS regain control of processor?
OS does this by introducing Internal and External events:
Internal events (things occurring within user process), such as:

- System call.

- Error (illegal instruction, addressing violation, etc.).
- Page fault.

These are also called *traps*. They all cause a state switch into the OS.
External events (things occurring outside the control of the user process):

- Character typed at terminal.
- Completion of disk operation (controller is ready for more work).
- Timer: to make sure OS eventually gets control.

External events are usually called *interrupts*. They all cause a state switch into the OS. *This means that user processes cannot directly take I/O interrupts*.

Structure of the Scheduler/Dispatcher

The generic structure of a scheduler/dispatcher is shown in figure 6. (below)

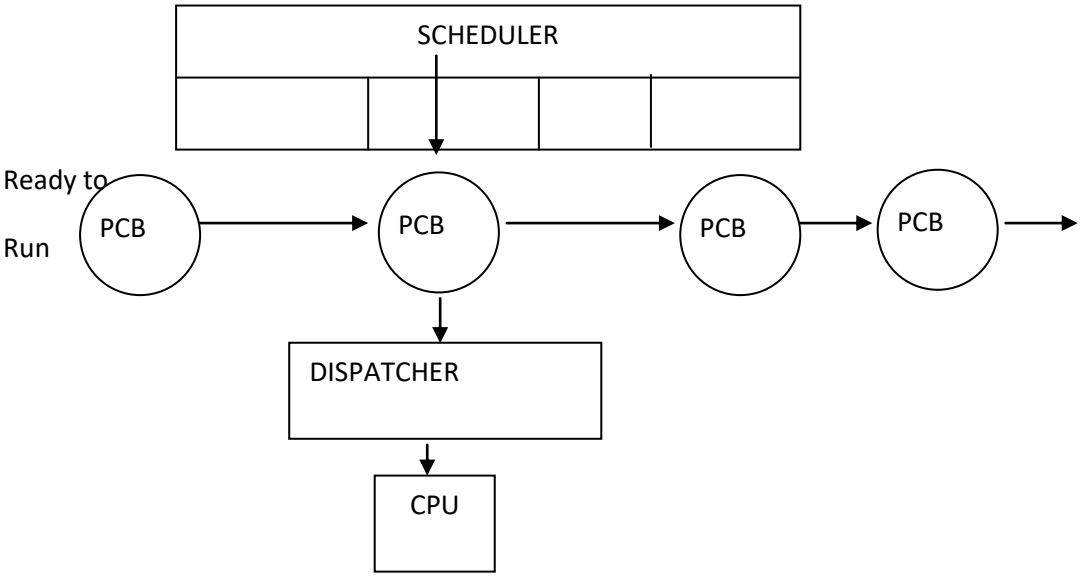


Figure 6 Structure of the Scheduler/Dispatcher

The scheduler is composed of a number of functional modules which perform the operations necessary for process management.

SCHEDULING

Available system resources range from CPU time to disk space to channel I/O time.
Resources fall into two classes:

- Preemptible: processor or I/O channel. Can take resource away, use it for something else, then give it back later.
- Non-preemptible: once given, it cannot be reused until process gives it back. Examples are file space, terminal, and maybe memory.

OS makes two related kinds of decisions about resources:

- Allocation: who gets what. Given a set of requests for resources, which processes should be given which resources in order to make most efficient use of the resources? Implication is that resources are not easily preemptible.
- Scheduling: how long can they keep it. When more resources are requested than can be granted immediately, in which order should they be serviced? Examples are processor scheduling (one processor, many processes), memory scheduling in virtual memory systems. Implication is that resource is preemptible.

CPU Scheduling

Processes may be in any one of three general scheduling states:

- Running. That is, using CPU time.
- Ready. That is, waiting for CPU time. Scheduler and dispatcher determine transitions between **ready** and **running** state.
- Blocked. Waiting for some other event: disk I/O, message, semaphore, etc. Transitions into and out of this state are caused by various processes.

There are two parts to CPU scheduling:

- The dispatcher provides the basic *mechanism* for running processes.
- The scheduler is a piece of OS code that decides the *priorities* of processes and how long each will run.

This is an example of policy/mechanism separation.

Goals for Scheduling Disciplines

- Efficiency of resource utilization *(keep CPU and disks busy)*.
- Minimize overhead *(context swaps)*.
- Minimize response time. (Define response time.)
- Distribute cycles equitably. What does this mean?

2.3 **Types of Schedulers**

There are three broad types of scheduler. These are **long-term, medium-term** and **short-term.** The **long-term** scheduler works with batch queue and selects the next batch job to be executed. **Medium-term** scheduler is in charge of handling the swapped-out or suspended processes. It works when the suspended condition is removed. **Short-term** scheduler allocates the processor among the pool of ready processes resident in memory. Figure 7 shows these schedulers.

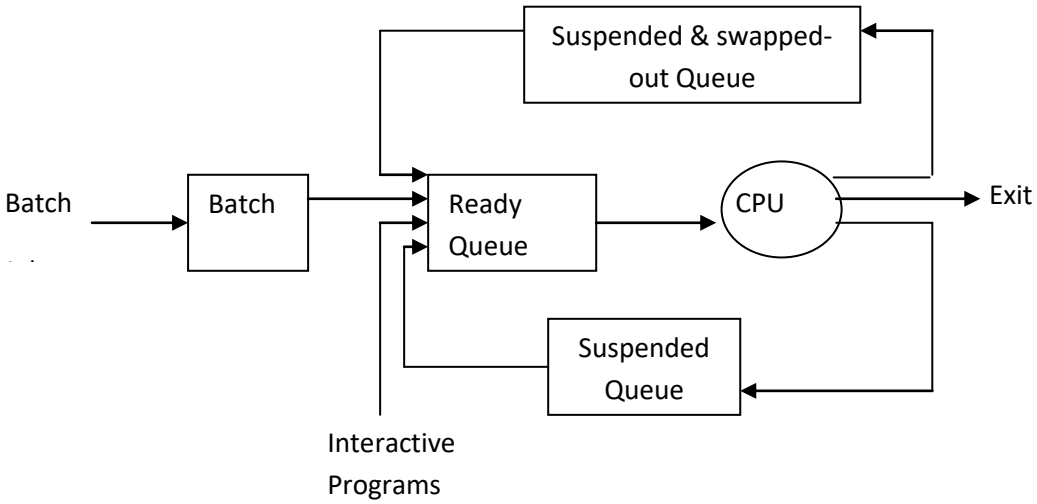


Figure 7 Types of schedulers

Preemptive and Non Preemptive Schedulers

There are two basic types of schedulers: Preemptive and Non preemptive.

Preemptive scheduler allows a running process to be interrupted by a higher priority process. In this case, the process is allowed to switch from running state to the ready state. Algorithms based preemptive are Round Robin, SRTF, Priority, etc.

In non preemptive scheduling, any new process has to wait until the running process finishes its allocated CPU cycle. Algorithms based on non preemptive are SJF, FCFS, etc.

Advantages of Preemptive scheduling:

(i) A process may not monopolize the CPU.
(ii) Average response time is improved.
(iii) Used in multiprogrammed environment.
(iv) CPU times are fairly utilized.

Disadvantages of Preemptive scheduling are:

(i) Suspending a running process and changing the context and dispatch new process takes more time.
(ii) Low priority processes have to wait if more higher priority processes are available.

Advantages of Non- Preemptive scheduling:

(i) It has minimal scheduling burden.
(ii) Very easy procedure.
(iii) Less computational resources are needed.
(iv) It has a high throughput rate.

Disadvantages of Non Preemptive scheduling are:

(i) A process may monopolize the CPU time.
(ii) Bugs in a process can cause a CPU to freeze up.
(iii) It does not favor multiprogramming.

Context Switching:

In computing, a context switch is **the process of storing the state of a process or thread**, so that it can be restored and resume execution at a later point. In a multitasking context, it refers to the process of storing the system state for one task, so that task can be paused and another task resumed.

The term dispatching is associated with **scheduling** and means roughly selecting the next task to run. So, in a typical task switch, for example due to a timer interrupt, the context switcher first saves the context of the interrupted task, establishes the context to run system code and then calls the dispatcher.

Example:

There are 4 processes in the ready queue, P1, P2, P3, P4; Their arrival times are t0, t1, t2, t3 respectively. A First-in First- out (FIFO) scheduling algorithm is used. Because P1 arrived first, the scheduler will decide it is the first process that should be executed, and the dispatcher will remove P1 from the ready queue and give it to the CPU. The scheduler will then determine P2 to be the next process that should be executed, so when the dispatcher returns to the queue for a new process, it will take P2 and give it to the CPU. This continues in the same way for P3, and then P4.

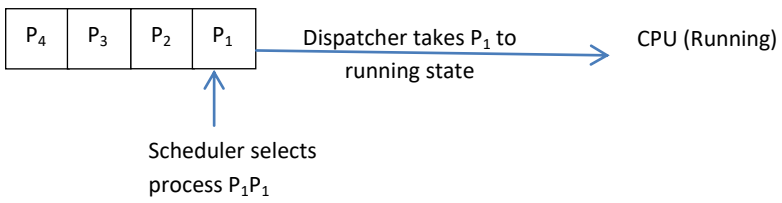


Figure 8: Example

Table 1: Comparison of dispatcher and scheduler

Properties	DISPATCHER	SCHEDULER
Definition:	Dispatcher is a module that gives control of CPU to the	Scheduler is something which

Properties	DISPATCHER	SCHEDULER
	process selected by short term scheduler	selects a process among various processes
Types:	There are no different types in dispatcher. It is just a code segment.	There are 3 types of schedulers i.e., Long-term, Short-term, Medium-term
Dependency:	Working of dispatcher is dependent on scheduler. Means dispatcher have to wait until scheduler selects a process.	Scheduler works independently. It works immediately when needed
Algorithm:	Dispatcher has no specific algorithm for its implementation	Scheduler works on various algorithm such as FCFS, SJF, RR etc.
Time Taken:	The time taken by dispatcher is called dispatch latency.	Time taken by scheduler is usually negligible. Hence, we neglect it.
Functions:	Dispatcher is also responsible for: Context Switching, Switch to user mode, jumping to proper location when process again restarted	

3. The role of interrupts

Interrupts are signals sent to the CPU by external devices, normally I/O devices. They **tell the CPU to stop its current activities and execute the appropriate part of the operating system**. Hardware Interrupts are generated by hardware devices to signal that they need some attention from the OS. There are three types of interrupts:

1. **Hardware Interrupts** are generated by hardware devices to signal that they need some attention from the OS. They may have just received some data (e.g., keystrokes on the keyboard or a data on the ethernet card); or they have just completed a task which the operating system previous requested, such as transferring data between the hard drive and memory.
2. **Software Interrupts** are generated by programs when they want to request a system call to be performed by the operating system.
3. **Traps** are generated by the CPU itself to indicate that some error or condition occurred for which assistance from the operating system is needed.

Interrupts are important because they give the user better control over the computer. Without interrupts, a user may have to wait for a given application to have a higher priority over the CPU to be ran. This ensures that the CPU will deal with the process immediately.

4. Concurrent execution: advantages and disadvantages

Concurrency is the execution of the multiple instruction sequences at the same time. It happens in the operating system when there are several process threads running in parallel. The running process threads always communicate with each other through shared memory or message passing. Concurrency results in sharing of resources which lead to problems like **deadlocks** and **resources starvation.** It helps in techniques like coordinating execution of processes, memory allocation and execution scheduling for maximizing throughput.

Principles of Concurrency

Both interleaved and overlapped processes can be viewed as examples of concurrent execution of processes, because they both present the same problems. The relative speed of execution cannot be predicted. It depends on the following:

- The activities of other processes;
- The way operating system handles interrupts;
- The scheduling policies of the operating system.

Problems in Concurrency:

- Sharing global resources: Sharing of global resources safely is difficult. If two processes both make use of a global variable and both perform read and write on that variable, then the order in which various read and write are executed is critical.
- Optimal allocation of resources: It is difficult for the operating system to manage the allocation of resources optimally.
- Locating programming errors: It is very difficult to locate a programming error because reports are usually not reproducible.
- Locking the channel: It may be inefficient for the operating system to simply lock the channel and prevents its use by other processes.

Advantages of Concurrency:

- Running of multiple applications: It enables running of multiple applications at the same time.
- Better resource utilization: It enables that the resources that are unused by one application can be used for other applications.
- Better average response time: Without concurrency, each application has to be run to completion before the next one can be run.
- Better performance: It enables better performance by the operating system. When one application uses only the processor and another application uses only the disk drive then the time to run both applications concurrently to completion will be shorter than the time to run each application consecutively.
- Reduced waiting time response time or turnaround time.
- Increased throughput or resource utilization.

Drawbacks of Concurrency

- It is required to protect multiple applications from one another.
- It is required to coordinate multiple applications through additional mechanisms.
- Additional performance overheads and complexities in operating systems are required for switching among applications.
- Sometimes running too many applications concurrently leads to severely degraded performance.

Issues of Concurrency:

- **Non-atomic:** Operations that are non-atomic but interruptible by multiple processes can cause problems.
- **Race conditions:** A race condition occurs if the outcome depends on which of several processes gets to a point first.
- **Blocking:** Processes can block waiting for resources. A process could be blocked for long period of time waiting for input from a terminal. If the process is required to periodically update some data, this would be very undesirable.
- **Starvation**: It occurs when a process does not obtain service to progress.
- **Deadlock**: It occurs when two processes are blocked and hence neither can proceed to execute.

What is a race condition? A race condition occur when there is an uncoordinated concurrent access to shared resources e.g memory, registers and devices, which leads to incorrect behavior of the program, deadlock or lost work.

An example: Two processes updating a counter simultaneously. Say the counter has a value of 1. Process A reads the variable but before updating Process B reads and updates the counter to 2. Process A, having no knowledge of what Process B did, updates the counter to 2. At this time, the correct behavior of the program would have the counter being incremented to 3 and not 2.

5. The mutual exclusion problem and some solutions

Mutual exclusion (MUTEX) in OS locks is a frequently used method for synchronizing processes or threads that want to access some shared resource. Their work justifies their name, if a thread operates on a resource, another thread that wants to do tasks on it must wait until the first one is done with its process. Mutex is a property of process synchronization which states that 'no two processes can exist in the critical section at any given point of time'. That is, only one process can be in the critical section at a time. **It must be free of deadlocks**: if processes are trying to enter the critical section, one of them must eventually be able to do so successfully, provided no process stays in the critical section permanently.

In this problem, there is a collection of asynchronous processes, each alternately executing a critical and a noncritical section that must be synchronized so that no two processes ever execute their critical sections concurrently or the mutual-exclusion solution to this makes the shared resource available only while the process is in a specific code segment called the critical section. It **controls access to the shared resource by controlling each mutual execution** of that part of its program where the resource would be used. Mutual exclusion is a property of concurrency control, which is instituted for the purpose of preventing race conditions.

6. Deadlock: causes, conditions, prevention

In a multi-process system, deadlock is an unwanted situation that arises in a shared resource environment, where a process indefinitely waits for a resource that is held by another process.

For example, assume a set of transactions $\{T_0, T_1, T_2, ...,T_n\}$. T_0 needs a resource X to complete its task. Resource X is held by T_1, and T_1 is waiting for a resource Y, which is held by T_2. T_2 is waiting for resource Z, which is held by T_0. Thus, all the processes wait for each other to release resources. In this situation, none of the processes can finish their task. At this point all processes are blocked and will remain so forever. This situation is known as a **deadlock**.

Deadlock can be defined formally as follows:

A set of processes is deadlocked if each process in the set is waiting for an event that only another process in the set can cause.

Deadlocks are not healthy for a system. In case a system is stuck in a deadlock, the transactions involved in the deadlock are either rolled back or restarted.

Assuming the OS detects the system is deadlocked, what can the OS do to recover from deadlock?

Answer:

(i) The OS can take resources if they are preemptive.

(ii) It can kill a process to free up resources.

(iii) It can try and take snapshots of process state, and roll back to state where the system was not deadlocked.

Deadlock Prevention

To prevent any deadlock situation in the system, the DBMS aggressively inspects all the operations, where transactions are about to execute. The DBMS inspects the operations and analyzes if they can create a deadlock situation. If it finds that a deadlock situation might occur, then that transaction is never allowed to be executed.

There are deadlock prevention schemes that use timestamp ordering mechanism of transactions in order to predetermine a deadlock situation.

Wait-Die Scheme

In this scheme, if a transaction requests to lock a resource (data item), which is already held with a conflicting lock by another transaction, then one of the two possibilities may occur −

- If $TS(T_i) < TS(T_j)$ − that is T_i, which is requesting a conflicting lock, is older than T_j − then T_i is allowed to wait until the data-item is available.
- If $TS(T_i) > TS(T_j)$ − that is T_i is younger than T_j − then T_i dies. T_i is restarted later with a random delay but with the same timestamp.

This scheme allows the older transaction to wait but kills the younger one.

Wound-Wait Scheme

In this scheme, if a transaction requests to lock a resource (data item), which is already held with conflicting lock by some another transaction, one of the two possibilities may occur −

- If $TS(T_i) < TS(T_j)$, then T_i forces T_j to be rolled back − that is T_i wounds T_j. T_j is restarted later with a random delay but with the same timestamp.
- If $TS(T_i) > TS(T_j)$, then T_i is forced to wait until the resource is available.

This scheme, allows the younger transaction to wait; but when an older transaction requests an item held by a younger one, the older transaction forces the younger one to abort and release the item.

In both the cases, the transaction that enters the system at a later stage is aborted.

Deadlock Avoidance

Aborting a transaction is not always a practical approach. Instead, deadlock avoidance mechanisms can be used to detect any deadlock situation in advance. Methods like "wait-for graph" are available but they are suitable for only those systems where transactions are lightweight having fewer instances of resource. In a bulky system, deadlock prevention techniques may work well.

Wait-for Graph

This is a simple method available to track if any deadlock situation may arise. For each transaction entering into the system, a node is created. When a transaction T_i requests for a lock on an item, say X, which is held by some other transaction T_j, a directed edge is created from T_i to T_j. If T_j releases item X, the edge between them is dropped and T_i locks the data item.

The system maintains this wait-for graph for every transaction waiting for some data items held by others. The system keeps checking if there's any cycle in the graph.

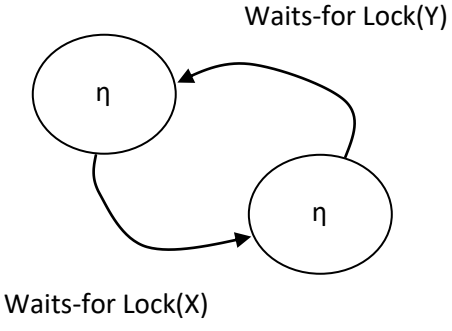


Figure 10

Here, we can use any of the two following approaches −

- First, do not allow any request for an item, which is already locked by another transaction. This is not always feasible and may cause starvation, where a transaction indefinitely waits for a data item and can never acquire it.
- The second option is to roll back one of the transactions. It is not always feasible to roll back the younger transaction, as it may be important than the older one. With the help of some relative algorithm, a transaction is chosen, which is to be aborted. This transaction is known as the **victim** and the process is known as **victim selection**.

The following four conditions must hold for there to be a deadlock

1. Mutual exclusion condition:- each resource is assigned to exactly one process.
2. Hold and wait condition:- process holding resources can request additional resources.
3. No preemption condition:- previously granted resources can't be forcibly taken away; only the process can voluntarily release resource.
4. Circular wait condition:- there must be a circular chain of two or more processes. Each of which is waiting for a resource held by the next member of the chain.

Deadlock arises because the number of units of a given resource is less than the number of processes requesting specified amounts of the resource. A system deadlock occurs when all processes in the system are incapable of further progress in execution. The simplest example occurs where two processes each hold a resource required by the other process and are unwilling to relinquish the resource they own.

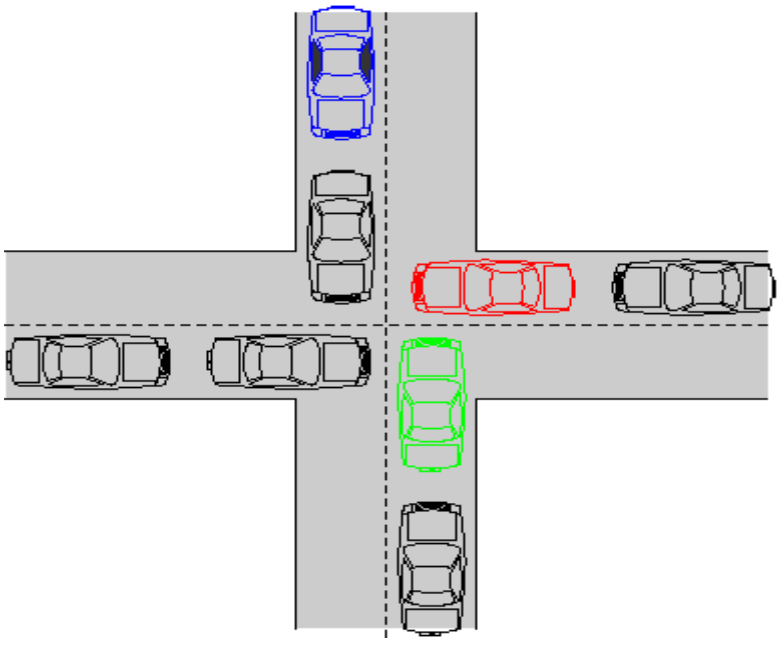


Figure 11 Deadlock example1

Deadlock defined: Deadlock may be defined as a situation where a process in each of a collection of processes is waiting for something from other processes in the collection. Since all are waiting, none can provide any of the things being waited for.

- Deadlock can occur over separate resources, as in above example, or even over separate copies of a single resource.

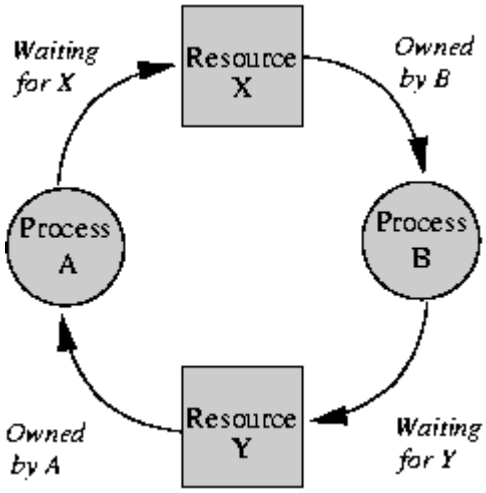
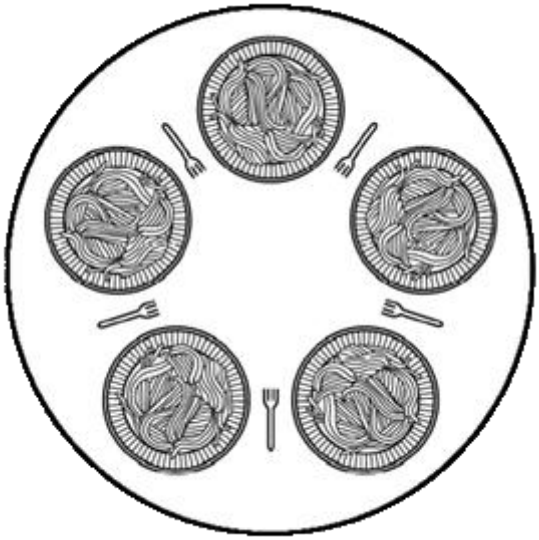


Figure 11 Deadlock example2

The **Dining Philosophers Problem** (due to Edsger Dijkstra and Tony Hoare, 1965)



- Five philosophers sit at a circular table doing one of two things: *eating* or *thinking*.
- While eating, they are not thinking, and while thinking, they are not eating.

- A large bowl of spaghetti is in the center and a fork is placed in between each pair of adjacent philosophers.
- This spaghetti is difficult to serve and eat with a single fork, so each philosopher **must acquire two forks to eat**.
- Each philosopher **can only use the forks on their immediate left and right**.
- Each philosopher can acquire forks only **one at a time**.
- Philosophers never speak to each other.

Solutions to the deadlock problem fall into two general categories:

- Detection: determine when the system is deadlocked and then take drastic action. Requires termination of one or more processes in order to release their resources. Usually this is not practical.
- Prevention: organize the system so that it is impossible for deadlock ever to occur. May lead to less efficient resource utilization in order to guarantee no deadlocks.

Deadlock prevention: must find a way to eliminate one of the four necessary conditions for deadlock:

In general, there are four conditions for deadlock:

- Limited access: resources cannot be shared.
- No preemption. Once given, a resource cannot be taken away.
- Multiple independent requests: processes do not ask for resources all at once.
- There is a circularity in the graph of who has what and who wants what. Draw graph showing processes as circles, resources as squares, arrows from process to resource waited for, from resource to owning process.

One mechanism of attacking deadlock is by trying to negate some of the conditions-deadlock avoidance.

Deadlock Avoidance

- Avoid deadlock by careful resource scheduling.
  - Requires that the system has some additional prior information available.
  - Simplest and most useful model requires that each process declare the maximum number of resources of each type that it may need.
  - The deadlock avoidance algorithm dynamically examines the resource allocation state to ensure that there can never be a circular wait.
  - Resource allocation state is defined by the number of available and allocated resources, and the maximum demands of the process.

Deadlock Prevention

- Prevent deadlock by resource scheduling so as to negate at least one of the four conditions
  - Attacking the mutual exclusion condition
  - Attacking the hold and wait condition
  - Attacking the no preemption condition
  - Attacking the circular wait condition

Banker's Algorithm and Its usefulness in Operating System (OS)

Banker's algorithm is used to **avoid deadlock** and **allocate resources** safely to each process in the computer system. The '**S-State'** examines all possible tests or activities before deciding whether the allocation should be allowed to each process. It also helps the operating system to successfully share the resources between all the processes. The banker's algorithm is named because it checks whether a person should be sanctioned a loan amount or not to help the bank system safely simulate allocation resources. In this section, we will learn the **Banker's Algorithm** in detail. Also, we will solve problems based on the **Banker's Algorithm**. To understand the Banker's Algorithm first we will see a real word example of it.

Suppose the number of account holders in a particular bank is '**n**', and the total money in a bank is '**T**'. If an account holder applies for a loan; first, the bank subtracts the loan amount from full cash and then estimates the cash difference is greater than T to approve the loan amount. These steps are taken because if another person applies for a loan or withdraws some amount from the bank, it helps the bank manage and operate all things without any restriction in the functionality of the banking system.

Similarly, it works in an **operating system**. When a new process is created in a computer system, the process must provide all types of information to the operating system like upcoming processes, requests for their resources, counting them, and delays. Based on these criteria, the operating system decides which process sequence should be executed or waited so that no deadlock occurs in a system. Therefore, it is also known as **deadlock avoidance algorithm** or **deadlock detection** in the operating system.

Advantages

The following are the essential characteristics of the Banker's algorithm: It contains various resources that meet the requirements of each process.

1. Each process should provide information to the operating system for upcoming resource requests, the number of resources, and how long the resources will be held.
2. It helps the operating system manage and control process requests for each type of resource in the computer system.
3. The algorithm has a Max resource attribute that represents the resources each process can hold the maximum number of resources in a system.

Disadvantages

1. It requires a fixed number of processes, and no additional processes can be started in the system while executing the process.
2. The algorithm does not allow the processes to exchange its maximum needs while processing its tasks.
3. Each process has to know and state their maximum resource requirement in advance for the system.
4. The number of resource requests can be granted in a finite time, but the time limit for allocating the resources is one year.

When working with a banker's algorithm, it requests to know about three things:

1. How much each process can request for each resource in the system. It is denoted by the [**MAX**] request.
2. How much each process is currently holding onto resource in a system. It is denoted by the [**ALLOCATED**] resource.

3. It represents the number of each resource currently available in the system. It is denoted by the [**AVAILABLE**] resource.

Following are the important data structures terminologies applied in the banker's algorithm:

Suppose **n** is the number of processes, and **m** is the number of each type of resources used in a computer system.

1. **Available**: It is an array of length 'm' that defines each type of resource available in the system. When Available R[j] = K, means that 'K' instances of Resources type R[j] are available in the system.
2. **Max:** It is a [n x m] matrix that indicates each process P[i] can store the maximum number of resources R[j] (each type) in a system.
3. **Allocation:** It is a matrix of m x n orders that indicates the type of resources currently allocated to each process in the system. When Allocation [i, j] = K, it means that process P[i] is currently allocated K instances of Resources type R[j] in the system.
4. **Need:** It is an M x N matrix sequence representing the number of remaining resources for each process. When the Need[i] [j] = k, then process P[i] may require K more instances of resources type Rj to complete the assigned work.
   Need[i][j] = Max[i][j] - Allocation[i][j].
5. **Finish**: It is the vector of the order **m**. It includes a Boolean value (true/false) indicating whether the process has been allocated to the requested resources, and all resources have been released after finishing its task.

The Banker's Algorithm is the combination of the safety algorithm and the resource request algorithm to control the processes and avoid deadlock in a system:

Safety Algorithm

It is a safety algorithm used to check whether or not a system is in a safe state or follows the safe sequence in a banker's algorithm:

1. There are two vectors **Work** and **Finish** of length m and n in a safety algorithm.

Initialize: Work = Available
Finish[i] = false; for I = 0, 1, 2, 3, 4… n - 1.

2. Check the availability status for each type of resources [i], such as:

Need[i] <= Work
Finish[i] == false
If the i does not exist, go to step 4.

3. Work = Work +Allocation(i) // to get new resource allocation

Finish[i] = true

Go to step 2 to check the status of resource availability for the next process.

4. If Finish[i] == true; it means that the system is safe for all processes.

Resource Request Algorithm

A resource request algorithm checks how a system will behave when a process makes each type of resource request in a system as a request matrix.

Let create a resource request array R[i] for each process P[i]. If the Resource Request$_i$ [j] equal to 'K', which means the process P[i] requires 'k' instances of Resources type R[j] in the system.

1. When the number of **requested resources** of each type is less than the **Need** resources, go to step 2 and if the condition fails, which means that the process P[i] exceeds its maximum claim for the resource. As the expression suggests:

If Request(i) <= Need
Go to step 2;

2. And when the number of requested resources of each type is less than the available resource for each process, go to step (3). As the expression suggests:

If Request(i) <= Available
Else Process P[i] must wait for the resource since it is not available for use.

3. When the requested resource is allocated to the process by changing state:

Available = Available - Request
Allocation(i) = Allocation(i) + Request (i)
Need$_i$ = Need$_i$ - Request$_i$

When the resource allocation state is safe, its resources are allocated to the process P(i). And if the new state is unsafe, the Process P (i) has to wait for each type of Request R(i) and restore the old resource-allocation state.

Example: Consider a system that contains five processes P1, P2, P3, P4, P5 and the three resource types A, B and C. Following are the resources types: A has 10, B has 5 and the resource type C has 7 instances.

Process	Allocation			Max			Available		
	A	B	C	A	B	C	A	B	C
P1	0	1	0	7	5	3	3	3	2
P2	2	0	0	3	2	2			
P3	3	0	2	9	0	2			
P4	2	1	1	2	2	2			
P5	0	0	2	4	3	3			

Answer the following questions using the banker's algorithm:

1. What is the reference of the need matrix?
2. Determine if the system is safe or not.

3. What will happen if the resource request (1, 0, 0) for process P1 can the system accept this request immediately?

Ans. 1: Context of the need matrix is as follows:

Need [i] = Max [i] - Allocation [i]
Need for P1: (7, 5, 3) - (0, 1, 0) = 7, 4, 3
Need for P2: (3, 2, 2) - (2, 0, 0) = 1, 2, 2
Need for P3: (9, 0, 2) - (3, 0, 2) = 6, 0, 0
Need for P4: (2, 2, 2) - (2, 1, 1) = 0, 1, 1
Need for P5: (4, 3, 3) - (0, 0, 2) = 4, 3, 1

Process	Need A	B	C
P1	7	4	3
P2	1	2	2
P3	6	0	0
P4	0	1	1
P5	4	3	1

Hence, we created the context of need matrix.

Ans. 2: Apply the Banker's Algorithm:
Available Resources of A, B and C are 3, 3, and 2.
Now we check if each type of resource request is available for each process.
Step 1: For Process P1:
Need <= Available
7, 4, 3 <= 3, 3, 2 condition is **false**.
So, we examine another process, P2.
Step 2: For Process P2:
Need <= Available
1, 2, 2 <= 3, 3, 2 condition **true**
New available = available + Allocation
(3, 3, 2) + (2, 0, 0) => 5, 3, 2
Similarly, we examine another process P3.
Step 3: For Process P3:
P3 Need <= Available
6, 0, 0 < = 5, 3, 2 condition is **false**.
Similarly, we examine another process, P4.
Step 4: For Process P4:
P4 Need <= Available
0, 1, 1 <= 5, 3, 2 condition is **true**
New Available resource = Available + Allocation
5, 3, 2 + 2, 1, 1 => 7, 4, 3

Similarly, we examine another process P5.

Step 5: For Process P5:
P5 Need <= Available
4, 3, 1 <= 7, 4, 3 condition is **true**
New available resource = Available + Allocation
7, 4, 3 + 0, 0, 2 => 7, 4, 5
Now, we again examine each type of resource request for processes P1 and P3.

Step 6: For Process P1:
P1 Need <= Available
7, 4, 3 <= 7, 4, 5 condition is **true**
New Available Resource = Available + Allocation
7, 4, 5 + 0, 1, 0 => 7, 5, 5

So, we examine another process P2.

Step 7: For Process P3:
P3 Need <= Available
6, 0, 0 <= 7, 5, 5 condition is true
New Available Resource = Available + Allocation
7, 5, 5 + 3, 0, 2 => 10, 5, 7

Hence, we execute the banker's algorithm to find the **safe state and the safe sequence:**

P2, P4, P5, P1 and P3.

Ans. 3: For granting the Request (1, 0, 2), first we have to check that **Request <= Available**, that is (1, 0, 2) <= (3, 3, 2), since the condition is true. So the process P1 gets the request immediately.

7. Models and mechanisms (semaphores, monitors, condition variables, rendezvous)

What are condition variables in OS?

Condition variables are synchronization primitives that enable threads to wait until a particular condition occurs. Condition variables are **user-mode objects that cannot be shared across processes**. Condition variables support operations that "wake one" or "wake all" waiting threads.

A rendezvous occurs **when two processes synchronize and subsequently exchange messages**. Rendezvous is symmetric, in that processes that wish to communicate both use the same primitive; and processes invoke Rendezvous with class designations, not procedure names.

Monitors are defined as **the construct of programming language**, which helps in controlling shared data access. The Monitor is a module or package which encapsulates shared data structure, procedures, and the synchronization between the concurrent procedure invocations.

Semaphores are **integer variables that are used to solve the critical section problem by using** two atomic operations, wait and signal that are used for process synchronization. The wait operation decrements the value of its argument S, if it is positive.

8. Producer-consumer problems and synchronization

8.1 The Producer-Consumer Problem

The Producer-Consumer problem is a classic problem that is used for multi-process synchronization i.e., synchronization between more than one processes. In the producer-consumer problem, there is one **Producer** that is producing something and there is one Consumer that is consuming the products produced by the Producer.

As an example of how these primitives can be used, let us consider the producer-consumer problem (also known as the bounded-buffer problem). Two processes share a common, fixed-size buffer. One of them, the producer, puts information into the buffer, and the other one, the consumer, takes it out. (It is also possible to generalize the problem to have **m** producers and **n** consumers, but we will consider only the case of one producer and one consumer because this assumption simplifies the solutions.)

Trouble arises when the producer wants to put a new item in the buffer, but it is already full. The solution is for the producer to go to sleep, to be awakened when the consumer has removed one or more items. Similarly, if the consumer wants to remove an item from the buffer and sees that the buffer is empty, it goes to sleep until the producer puts something in the buffer and wakes it up. This approach sounds simple enough, but it leads to the same kinds of race conditions we saw earlier with the spooler directory. To keep track of the number of items in the buffer, we will need a variable, count. If the maximum number of items the buffer can hold is N, the producer's code will first test to see if count is N. If it is, the producer will go to sleep; if it is not, the producer will add an item and increment count. The consumer's code is similar: first test count to see if it is 0. If it is, go to sleep; if it is nonzero, remove an item and decrement the counter. Each of the processes also tests to see if the other should be awakened, and if so, wakes it up.

To express system calls such as sleep and wakeup in C, we will show them as calls to library routines. They are not part of the standard C library but presumably would be made available on any system that actually had these system calls. The procedures insert item and remove item, which are not shown. It handles the bookkeeping of putting items into the buffer and taking items out of the buffer. Now let us get back to the **race condition**.

A race condition is a situation that may occur inside a critical section. This happens when the result of multiple thread execution in critical section differs according to the order in which the threads execute.

A race condition occurs when two threads access a shared variable at the same time. The first thread reads the variable, and the second thread reads the same value from the variable. Then the first thread and second thread perform their operations on the value, and they race to see which thread can write the value last to the shared variable. The value of the thread that writes its value last is preserved, because the thread is writing over the value that the previous thread wrote. For example:

Thread1: Total =Total + val1

Thread2: Total =Total – val2

In this example, the value of the thread2 that writes its value last is preserved, because the thread2 is writing over the value that the previous thread1 wrote.

What is the problem here?

The following are the problems that might occur in the Producer-Consumer:

The producer should produce data only when the buffer is not full. If the buffer is full, then the producer shouldn't be allowed to put any data into the buffer.

The consumer should consume data only when the buffer is not empty. If the buffer is empty, then the consumer shouldn't be allowed to take any data from the buffer.

The producer and consumer should not access the buffer at the same time.

What is the solution?

The above three problems can be solved with the help of semaphores
In the producer-consumer problem, we use three semaphore variables:

Semaphore S: This semaphore variable is used to achieve mutual exclusion between processes. By using this variable, either Producer or Consumer will be allowed to use or access the shared buffer at a particular time. This variable is set to 1 initially.

Semaphore E: This semaphore variable is used to define the empty space in the buffer. Initially, it is set to the whole space of the buffer i.e., "n" because the buffer is initially empty.

Semaphore F: This semaphore variable is used to define the space that is filled by the producer. Initially, it is set to "0" because there is no space filled by the producer initially.

By using the above three semaphore variables and by using the wait () and signal () function, we can solve our problem (the wait () function decreases the semaphore variable by 1 and the signal () function increases the semaphore variable by 1). So. let's see how.

The following is the pseudo-code for the producer:

```
void producer () {
   while(T) {
      produce ()
      wait(E)
      wait(S)
      append ()
      signal(S)
      signal(F)
   }
}
```

The above code can be summarized as:
while () is used to produce data, again and again, if it wishes to produce, again and again.
produce () function is called to produce data by the producer.

wait(E) will reduce the value of the semaphore variable "E" by one i.e., when the producer produces something then there is a decrease in the value of the empty space in the buffer. If the buffer is full i.e., the value of the semaphore variable "E" is "0", then the program will stop its execution and no production will be done.
wait(S) is used to set the semaphore variable "S" to "0" so that no other process can enter into the critical section.

append () function is used to append the newly produced data in the buffer.

signal(s) is used to set the semaphore variable “S” to “1” so that other processes can come into the critical section now because the production is done and the append operation is also done.

signal(F) is used to increase the semaphore variable “F” by one because after adding the data into the buffer, one space is filled in the buffer and the variable “F” must be updated.

This is how we solve the produce part of the producer-consumer problem. Now, let’s see the consumer solution. The following is the code for the consumer:

```
void consumer () {
   while(T) {
      wait(F)
      wait(S)
      take ()
      signal(S)
      signal(E)
      use ()
   }
}
```

The above code can be summarized as:

while () is used to consume data, again and again, if it wishes to consume, again and again.
wait(F) is used to decrease the semaphore variable “F” by one because if some data is consumed by the consumer, then the variable “F” must be decreased by one.
wait(S) is used to set the semaphore variable “S” to “0” so that no other process can enter into the critical section.
take () function is used to take the data from the buffer by the consumer.
signal(S) is used to set the semaphore variable “S” to “1” so that other processes can come into the critical section now because the consumption is done and the take operation is also done.
signal(E) is used to increase the semaphore variable “E” by one because after taking the data from the buffer, one space is freed from the buffer and the variable “E” must be increased.
use () is a function that is used to use the data taken from the buffer by the process to do some operation.
So, this is how we can solve the producer-consumer problem.

8.2 Synchronization

Synchronization mechanism is intended for groups of processes rather than two-process producer-consumer type situations. Some applications are divided into phases and have the rule that no process may proceed into the next phase until all processes are ready to proceed to the next phase. This behavior may be achieved by placing a barrier at the end of each phase. When a process reaches the barrier, it is blocked until all processes have reached the barrier. This allows groups of processes to synchronize.

9.0 Multiprocessor issues (spin-locks, reentrancy)

What is spinlock?

Spinlock: Is a locking system mechanism. It allows a thread to acquire it to simply wait in loop until the lock is available, i.e., a thread waits in a loop or spin until the lock is available. Spinlock is held for a short period of time. Spinlock are useful in multiprocessor system. The thread will wait in a loop, and then continuously judge whether the lock can be acquired successfully. It will not exit the loop until the lock is acquired. The thread that gets the lock is always active, but it doesn’t perform any effective task. Using this lock will cause a lot of problems busy-waiting.

The problems of spin lock: There is one problem when using spin lock
1. If a thread holds the lock for a long time, it will cause other threads waiting for the lock to enter the cycle waiting and consume CPU. Improper use will result in high CPU utilization.
2. The spin lock implemented by Java above is not fair, that is, it cannot satisfy the priority of the thread with the longest waiting time to obtain the lock. Unfair locks can lead to "thread starvation".

Advantages of spin lock

- Spin lock will not make the thread state switch, always in the user state, that is, the thread is always active; it will not make the thread into the blocking state, reduce unnecessary context switching, and the execution speed is fast.
- When the lock cannot be acquired, the non spin lock will enter the blocking state and enter the kernel state. When the lock is acquired, it needs to recover from the kernel state and need to switch the thread context. (When the thread is blocked, it will enter the kernel (Linux) scheduling state, which will cause the system to switch back and forth between the user state and the kernel state, seriously affecting the performance of the lock.)

Reentrant spin lock and non-reentrant spin lock

A careful analysis of the code at the beginning of the article shows that it does not support reentry. That is, when a thread has acquired the lock for the first time, it reacquires the lock again before the lock is released, and it cannot acquire it successfully for the second time. Because CAS is not satisfied, the second acquisition will enter the while loop waiting. If it is a reentrant lock, the second acquisition should be successful. Moreover, even if it can be acquired successfully for the second time, when the lock is released for the first time, the lock acquired for the second time will also be released, which is unreasonable.

In software engineering, a spinlock is a lock which causes a thread trying to acquire it to simply wait in a loop ("spin") while repeatedly checking if the lock is available. The longer a thread holds a lock, the greater the risk that the thread will be interrupted by the OS scheduler while holding the lock. In computing, a computer program or subroutine is called reentrant if multiple invocations can safely run concurrently on multiple processors, or on a single processor system, where a reentrant procedure can be interrupted in the middle of its execution and then safely be called again ("re-entered") before its previous.

10.0 Memory Management System

Memory management is the functionality of an operating system which handles or manages primary memory and moves processes back and forth between main memory and disk during execution. Memory management keeps track of each and every memory location, regardless of either it is allocated to some process or it is free. It checks how much memory is to be allocated to processes. It decides which process will get memory at what time. It tracks whenever some memory gets freed or unallocated and correspondingly it updates the status.

Process Address Space

The process address space is the set of logical addresses that a process references in its code. For example, when 32-bit addressing is in use, addresses can range from 0 to 0x7fffffff; that is, 2^31 possible numbers, for a total theoretical size of 2 gigabytes.

The operating system takes care of mapping the logical addresses to physical addresses at the time of memory allocation to the program. There are three types of addresses used in a program before and after memory is allocated:

S.N.	Memory Addresses & Description
1	**Symbolic addresses** The addresses used in a source code. The variable names, constants, and instruction labels

	are the basic elements of the symbolic address space.
2	**Relative addresses** At the time of compilation, a compiler converts symbolic addresses into relative addresses.
3	**Physical addresses** The loader generates these addresses at the time when a program is loaded into main memory.

Virtual and physical addresses are the same in compile-time and load-time address-binding schemes. Virtual and physical addresses differ in execution-time address-binding scheme.

The set of all logical addresses generated by a program is referred to as a **logical address space**. The set of all physical addresses corresponding to these logical addresses is referred to as a **physical address space.**

The runtime mapping from virtual to physical address is done by the memory management unit (MMU) which is a hardware device. MMU uses following mechanism to convert virtual address to physical address.

- The value in the base register is added to every address generated by a user process, which is treated as offset at the time it is sent to memory. For example, if the base register value is 10000, then an attempt by the user to use address location 100 will be dynamically reallocated to location 10100.
- The user program deals with virtual addresses; it never sees the real physical addresses.

Static vs Dynamic Loading

The choice between Static or Dynamic Loading is to be made at the time of computer program being developed. If you have to load your program statically, then at the time of compilation, the complete programs will be compiled and linked without leaving any external program or module dependency. The linker combines the object program with other necessary object modules into an absolute program, which also includes logical addresses.

If you are writing a Dynamically loaded program, then your compiler will compile the program and for all the modules which you want to include dynamically, only references will be provided and rest of the work will be done at the time of execution.

At the time of loading, with **static loading**, the absolute program (and data) is loaded into memory in order for execution to start.

If you are using **dynamic loading**, dynamic routines of the library are stored on a disk in relocatable form and are loaded into memory only when they are needed by the program.

Static vs Dynamic Linking

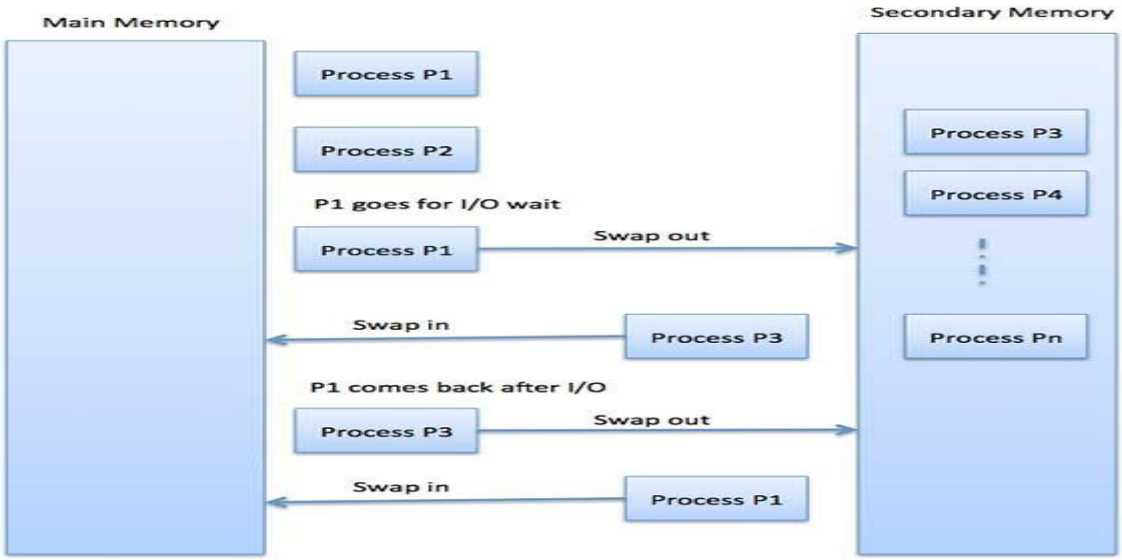
As explained above, when static linking is used, the linker combines all other modules needed by a program into a single executable program to avoid any runtime dependency.

When dynamic linking is used, it is not required to link the actual module or library with the program, rather a reference to the dynamic module is provided at the time of compilation and linking. Dynamic Link Libraries (DLL) in Windows and Shared Objects in Unix are good examples of dynamic libraries.

Swapping

Swapping is a mechanism in which a process can be swapped temporarily out of main memory (or move) to secondary storage (disk) and make that memory available to other processes. At some later time, the system swaps back the process from the secondary storage to main memory.

Though performance is usually affected by swapping process but it helps in running multiple and big processes in parallel and that's the reason **Swapping is also known as a technique for memory compaction**.



The total time taken by swapping process includes the time it takes to move the entire process to a secondary disk and then to copy the process back to memory, as well as the time the process takes to regain main memory.

Let us assume that the user process is of size 2048KB and on a standard hard disk where swapping will take place has a data transfer rate around 1 MB per second. The actual transfer of the 1000K process to or from memory will take

2048KB / 1024KB per second

= 2 seconds

= 2000 milliseconds

Now considering in and out time, it will take complete 4000 milliseconds plus other overhead where the process competes to regain main memory.

Memory Allocation

Main memory usually has two partitions:

- **Low Memory** − Operating system resides in this memory.
- **High Memory** − User processes are held in high memory.

Operating system uses the following memory allocation mechanism.

S.N.	Memory Allocation & Description
1	**Single-partition allocation** In this type of allocation, relocation-register scheme is used to protect user processes from each other, and from changing operating-system code and data. Relocation register contains value of smallest physical address whereas limit register contains range of logical addresses. Each logical address must be less than the limit register.
2	**Multiple-partition allocation** In this type of allocation, main memory is divided into a number of fixed-sized partitions where each partition should contain only one process. When a partition is free, a process is selected from the input queue and is loaded into the free partition. When the process terminates, the partition becomes available for another process.

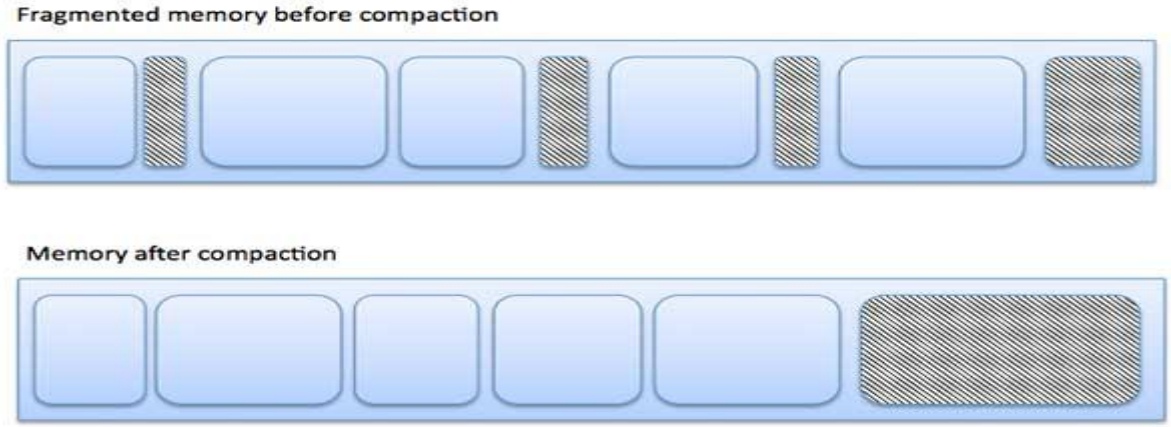
Fragmentation

As processes are loaded and removed from memory, the free memory space is broken into little pieces. It happens after sometimes that processes cannot be allocated to memory blocks considering their small size and memory blocks remains unused. This problem is known as Fragmentation.

Fragmentation is of two types:

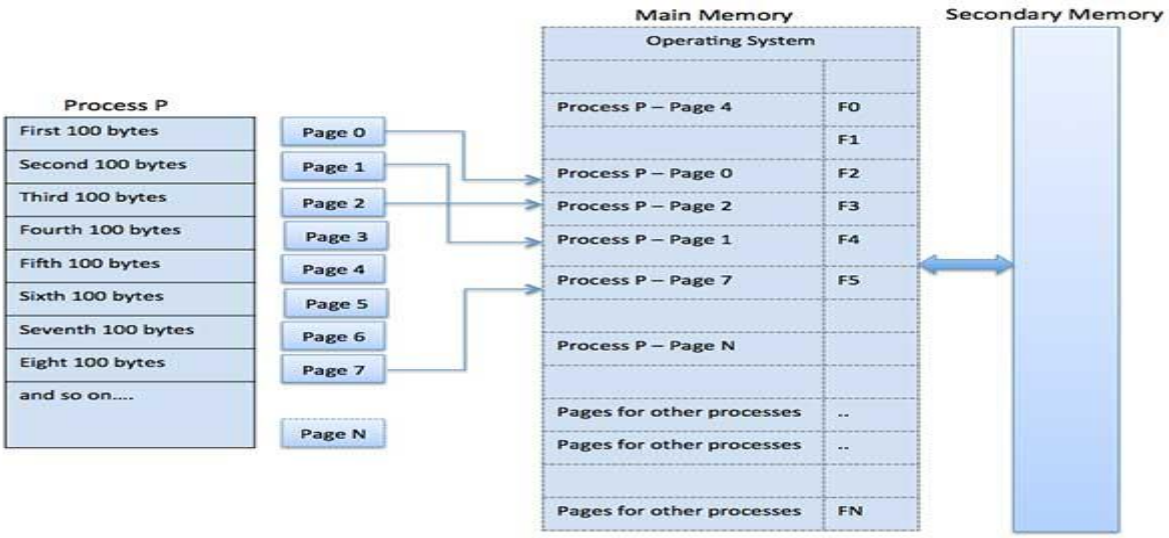
S.N.	Fragmentation & Description
1	**External fragmentation** Total memory space is enough to satisfy a request or to reside a process in it, but it is not contiguous, so it cannot be used.
2	**Internal fragmentation** Memory block assigned to process is bigger. Some portion of memory is left unused, as it cannot be used by another process.

The following diagram shows how fragmentation can cause waste of memory and a compaction technique can be used to create more free memory out of fragmented memory −



External fragmentation can be reduced by compaction or shuffle memory contents to place all free memory together in one large block. To make compaction feasible, relocation should be dynamic.
The internal fragmentation can be reduced by effectively assigning the smallest partition but large enough for the process.

Paging
A computer can address more memory than the amount physically installed on the system. This extra memory is actually called virtual memory and it is a section of a hard that's set up to emulate the computer's RAM. Paging technique plays an important role in implementing virtual memory.
Paging is a memory management technique in which process address space is broken into blocks of the same size called **pages** (size is power of 2, between 512 bytes and 8192 bytes). The size of the process is measured in the number of pages.
Similarly, main memory is divided into small fixed-sized blocks of (physical) memory called **frames** and the size of a frame is kept the same as that of a page to have optimum utilization of the main memory and to avoid external fragmentation.



Address Translation

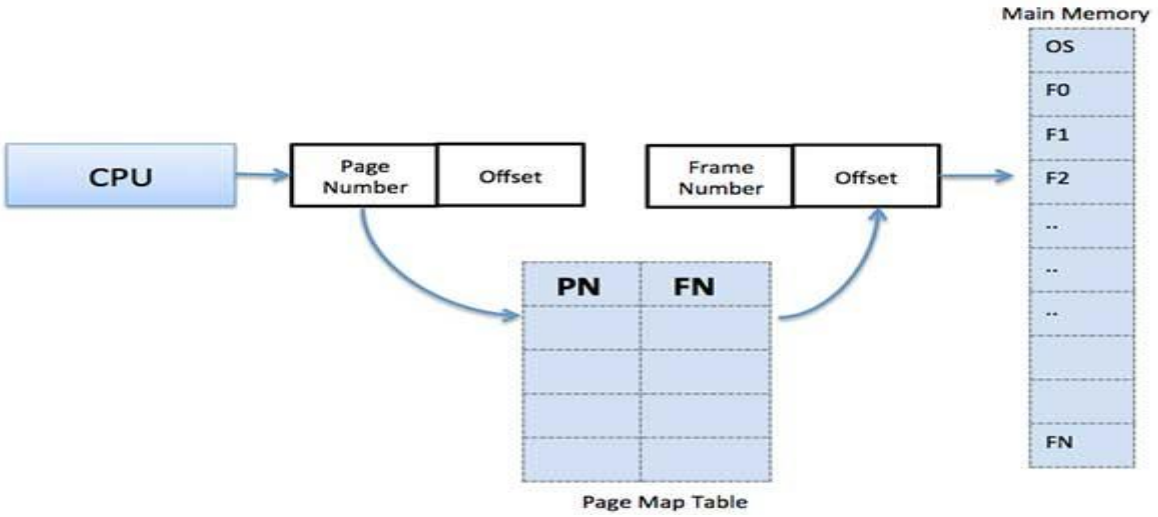
Page address is called **logical address** and represented by **page number** and the **offset**.

Logical Address = Page number + page offset

Frame address is called **physical address** and represented by a **frame number** and the **offset**.

Physical Address = Frame number + page offset

A data structure called **page map table** is used to keep track of the relation between a page of a process to a frame in physical memory.



When the system allocates a frame to any page, it translates this logical address into a physical address and create entry into the page table to be used throughout execution of the program.

When a process is to be executed, its corresponding pages are loaded into any available memory frames. Suppose you have a program of 8Kb but your memory can accommodate only 5Kb at a given point in time, then the paging concept will come into picture. When a computer runs out of RAM, the operating system (OS) will move idle or unwanted pages of memory to secondary memory to free up RAM for other processes and brings them back when needed by the program.

This process continues during the whole execution of the program where the OS keeps removing idle pages from the main memory and write them onto the secondary memory and bring them back when required by the program.

Solved example:

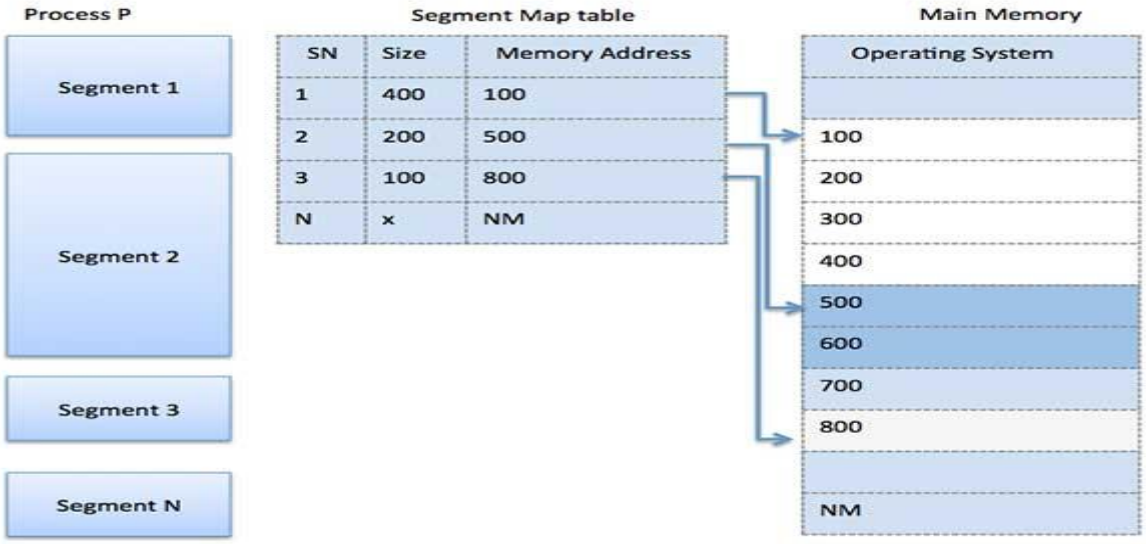
i) Assume the page table pointer contains 4096 and the page number is 128. What is the starting address of the block which specifies the information for page #128? Suppose the page is already in memory and has a starting address of 65,536. What is the physical address given the word number is 32? How many pages of 2048 bytes each can we accommodate in a 256 KB memory bank?

Solution:
The sought address within the page table is 4096 +128 = 4224.
The Physical address corresponding to the given page number #128 is 65536 + 32 =65,568.
The bank in question can accommodate a total of 256/2 =128 pages.

Advantages and Disadvantages of Paging
Here is a list of advantages and disadvantages of paging:

- Paging reduces external fragmentation, but still suffer from internal fragmentation.
- Paging is simple to implement and assumed as an efficient memory management technique.
- Due to equal size of the pages and frames, swapping becomes very easy.
- Page table requires extra memory space, so may not be good for a system having small RAM.

Segmentation
Segmentation is a memory management technique in which each job is divided into several segments of different sizes, one for each module that contains pieces that perform related functions. Each segment is actually a different logical address space of the program.
When a process is to be executed, its corresponding segmentation are loaded into non-contiguous memory though every segment is loaded into a contiguous block of available memory.
Segmentation memory management works very similar to paging but here segments are of variable-length where as in paging pages are of fixed size.
A program segment contains the program's main function, utility functions, data structures, and so on. The operating system maintains a **segment map table** for every process and a list of free memory blocks along with segment numbers, their size and corresponding memory locations in main memory. For each segment, the table stores the starting address of the segment and the length of the segment. A reference to a memory location includes a value that identifies a segment and an offset.



Combined Paging and Segmentation
Both segmented and paged implementations have their respective advantages and disadvantages, and neither is superior to the other over all characteristics. Some computer systems combine the two approaches in order to enjoy the benefits of both. One popular approach is to use segmentation from the user's point of view but to divide each segment into pages of fixed size for purposes of allocation. In this way, the combined system retains most of the advantages of segmentation. The principle of address translation in combined segmentation and paging systems is illustrated below.

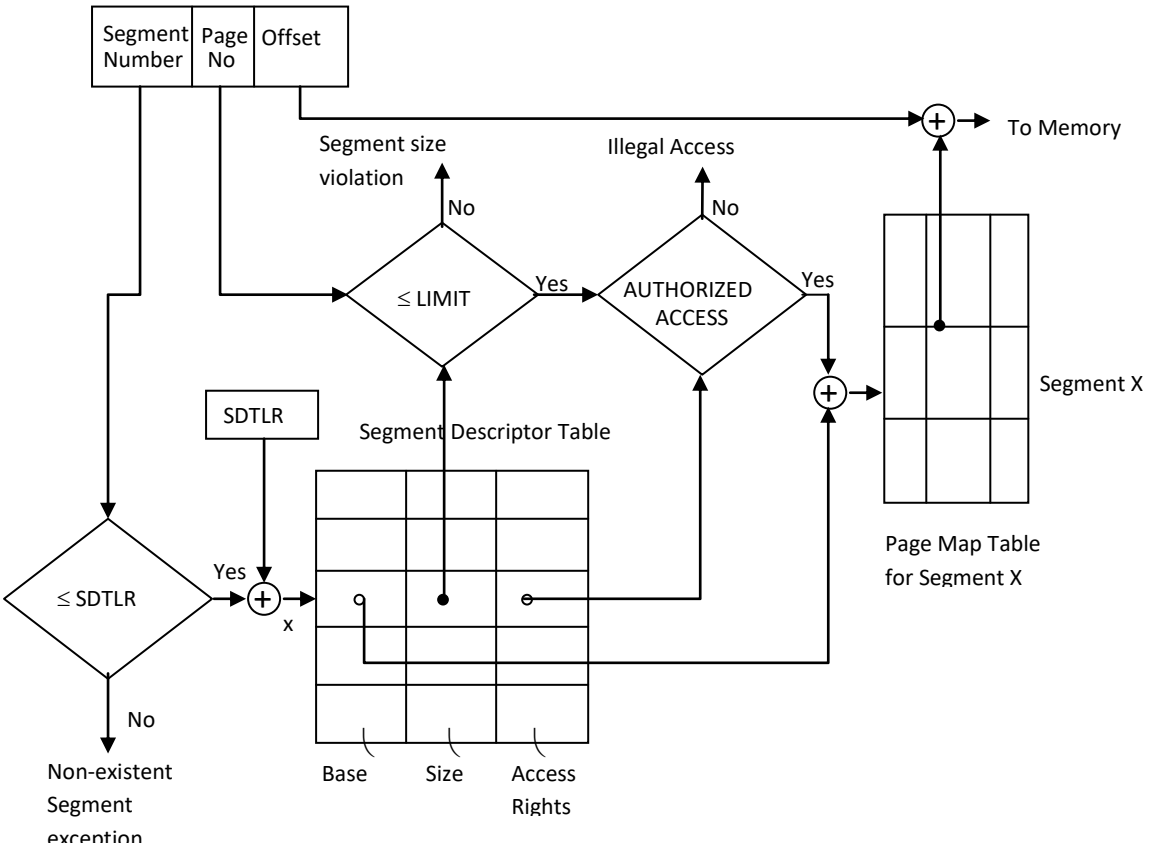


Figure: Segmentation and Paging

SDTLR – Segment descriptor Table Limit Register

As shown, both segment descriptor tables and page-map tables are required for mapping. Instead of containing the base and limit of the corresponding segment, each entry of the SDT contains the base address and size of the page-map table to be used for mapping of the related segment's pages. The presence bit in each PMT entry indicates whether the corresponding page is in the real memory or not.

Access rights are usually recorded as a part of segment descriptors. Each virtual address in a combined system basically consists of three fields: the segment number, the page number and the offset within the page. When a virtual address is presented to the mapping hardware, the segment number is used to locate the corresponding page-map table. Provided that the issuing process is authorized to make the intended type of reference to the target segment, the page number is used to index the PMT as usual.

Virtual Memory System

A computer can address more memory than the amount physically installed on the system. This extra memory is actually called **virtual memory** and it is a section of a hard disk that's set up to emulate the computer's RAM.

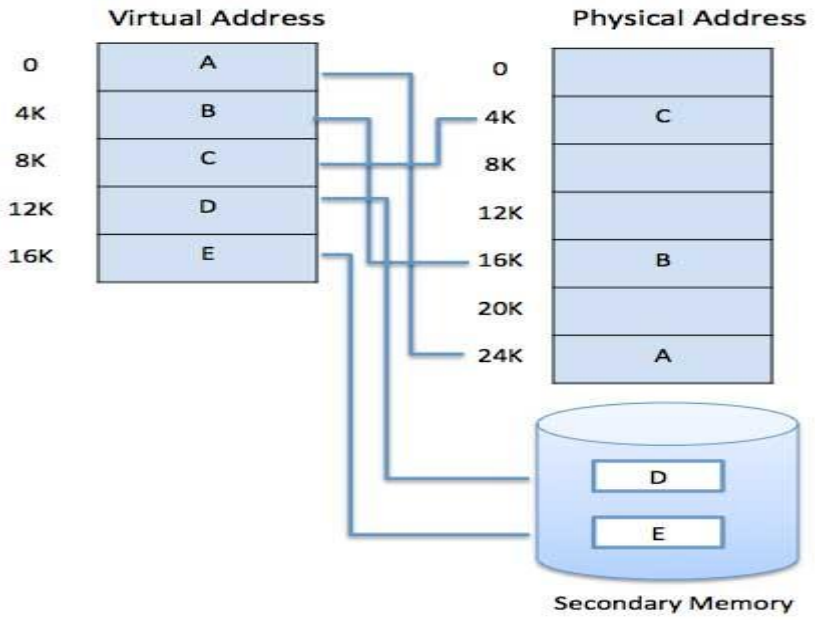
The main visible advantage of this scheme is that programs can be larger than physical memory. Virtual memory serves two purposes. First, it allows us to extend the use of physical memory by using disk. Second, it allows us to have memory protection, because each virtual address is translated to a physical address.

Following are the situations, when entire program is not required to be loaded fully in main memory.

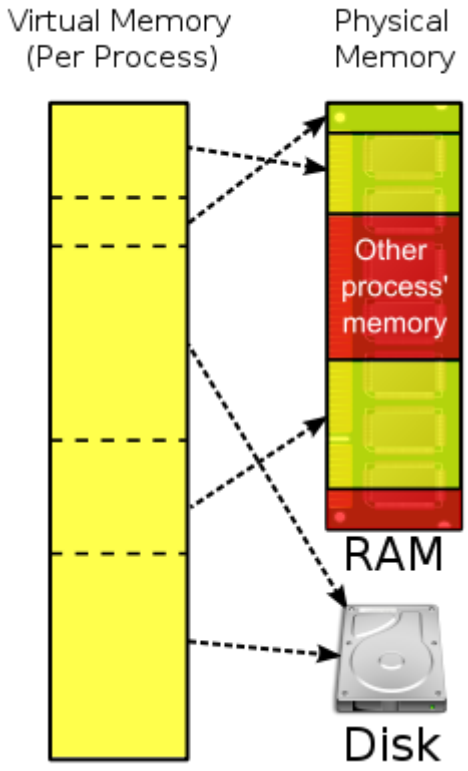
- User written error handling routines are used only when an error occurred in the data or computation.
- Certain options and features of a program may be used rarely.
- Many tables are assigned a fixed amount of address space even though only a small amount of the table is actually used.
- The ability to execute a program that is only partially in memory would counter many benefits.
- Less number of I/O would be needed to load or swap each user program into memory.
- A program would no longer be constrained by the amount of physical memory that is available.

- Each user program could take less physical memory, more programs could be run the same time, with a corresponding increase in CPU utilization and throughput.

Modern microprocessors intended for general-purpose use, a memory management unit, or MMU, is built into the hardware. The MMU's job is to translate virtual addresses into physical addresses. A basic example is given below −



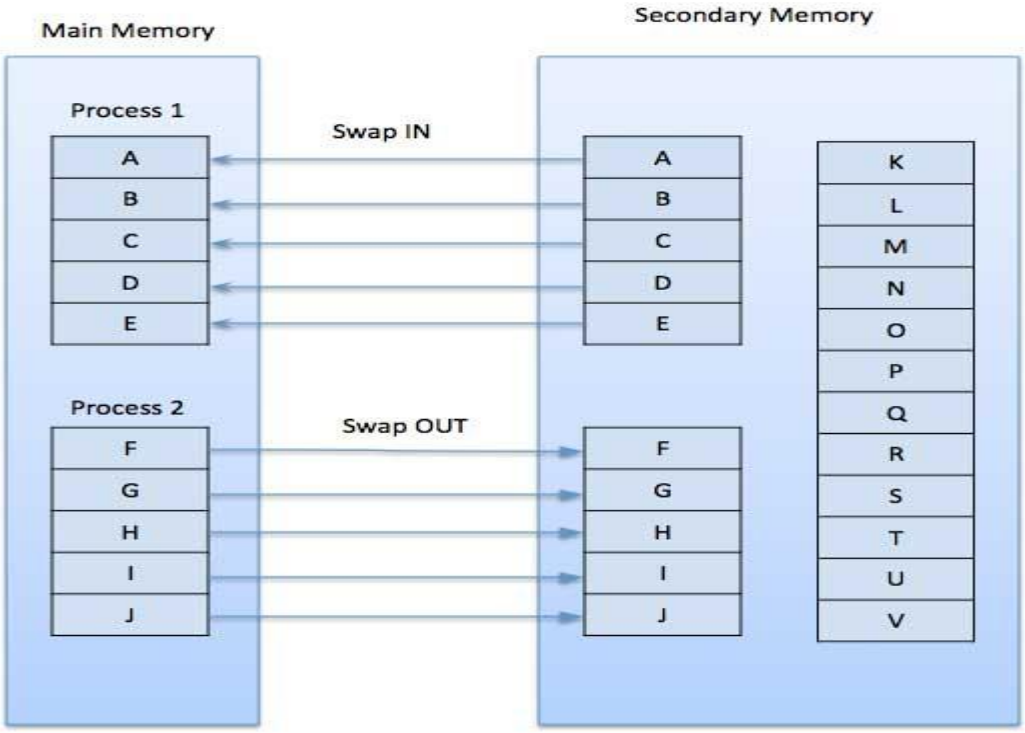
Virtual memory is commonly implemented by demand paging. It can also be implemented in a segmentation system. Demand segmentation can also be used to provide virtual memory.



Demand Paging

A demand paging system is quite similar to a paging system with swapping where processes reside in secondary memory and pages are loaded only on demand, not in advance. When a context switch occurs, the

operating system does not copy any of the old program's pages out to the disk or any of the new program's pages into the main memory Instead, it just begins executing the new program after loading the first page and fetches that program's pages as they are referenced.



While executing a program, if the program references a page which is not available in the main memory because it was swapped out a little ago, the processor treats this invalid memory reference as a **page fault** and transfers control from the program to the operating system to demand the page back into the memory.

Advantages

Following are the advantages of Demand Paging −

- Large virtual memory.
- More efficient use of memory.
- There is no limit on degree of multiprogramming.

Disadvantages

- Number of tables and the amount of processor overhead for handling page interrupts are greater than in the case of the simple paged management techniques.

Page Replacement Algorithm

Page replacement algorithms are the techniques by which an Operating System decides which memory pages to swap out, write to disk when a page of memory needs to be allocated. Paging happens whenever a page fault occurs and a free page cannot be used for allocation purpose accounting to reason that pages are not available or the number of free pages is lower than required pages.

When the page that was selected for replacement and was paged out, is referenced again, it has to read in from disk, and this requires for I/O completion. This process determines the quality of the page replacement algorithm: the lesser the time waiting for page-ins, the better is the algorithm.

A page replacement algorithm looks at the limited information about accessing the pages provided by hardware, and tries to select which pages should be replaced to minimize the total number of page misses, while balancing it with the costs of primary storage and processor time of the algorithm itself. There are many different page replacement algorithms. We evaluate an algorithm by running it on a particular **string of memory reference** and computing the number of page faults,

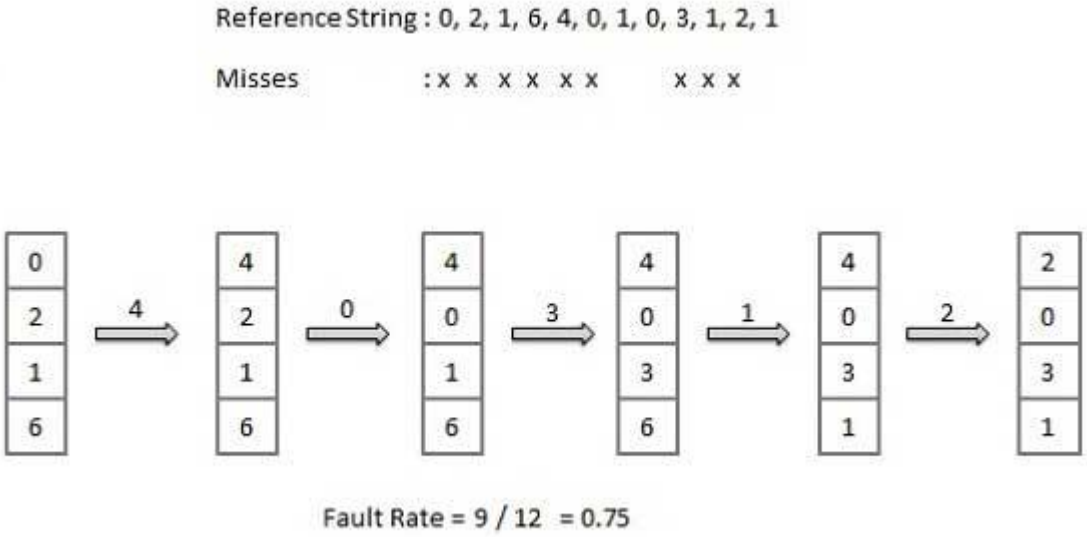
Reference String

The string of memory references is called reference string. Reference strings are generated artificially or by tracing a given system and recording the address of each memory reference. The latter choice produces a large number of data, where we note two things.

- For a given page size, we need to consider only the page number, not the entire address.
- If we have a reference to a page **p**, then any immediately following references to page **p** will never cause a page fault. Page p will be in memory after the first reference; the immediately following references will not fault.
- For example, consider the following sequence of addresses: 123,215,600,1234,76,96
- If page size is 100, then the reference string is 1,2,6,12,0,0

First In First Out (FIFO) algorithm

- Oldest page in main memory is the one which will be selected for replacement.
- Easy to implement, keep a list, replace pages from the tail and add new pages at the head.



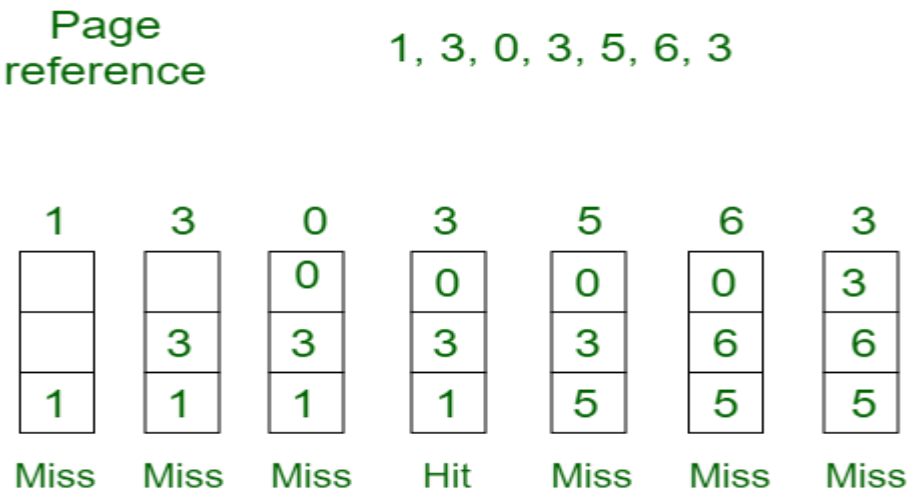
What is a Page Fault? A page fault happens when a running program accesses a memory page that is mapped into the virtual address space but not loaded in physical memory. Since actual physical memory is much smaller than virtual memory, page faults happen. In case of a page fault, Operating System might have to replace one of the existing pages with the newly needed page. Different page replacement algorithms suggest different ways to decide which page to replace. The target for all algorithms is to reduce the number of page faults.

Note that: Pages Not Found = Page_ Fault (or Miss), while Pages Found = Page_ Hit

Page Replacement Algorithms:

1. First In First Out (FIFO): This is the simplest page replacement algorithm. In this algorithm, the operating system keeps track of all pages in the memory in a queue, the oldest page is in the front of the queue. When a page needs to be replaced page in the front of the queue is selected for removal.

Example 1: Consider page reference string 1, 3, 0, 3, 5, 6, 3 with 3 page frames.Find the number of page faults.



Total Page Fault = 6

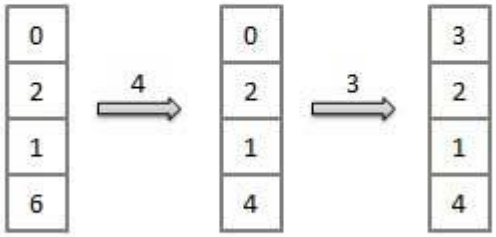
Initially, all slots are empty, so when 1, 3, 0 came they are allocated to the empty slots —> **3 Page Faults. (This is called page Miss).**
When 3 comes, it is already in memory so —> **0 Page Faults (this is called page Hit).** Then 5 comes, it is not available in memory so it replaces the oldest page slot i.e 1. —>**1 Page Fault.** When 6 comes, it is also not available in memory so it replaces the oldest page slot i.e 3 —>**1 Page Fault.** Finally, when 3 come it is not available so it replaces the oldest 0 —> **1 page fault.**
Giving a total page fault =6.

Optimal Page algorithm

- An optimal page-replacement algorithm has the lowest page-fault rate of all algorithms. An optimal page-replacement algorithm exists, and has been called OPT or MIN.
- Replace the page that will not be used for the longest period of time. Use the time when a page is to be used.

Reference String : 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1

Misses : x x x x x x



Fault Rate = 6 / 12 = 0.50

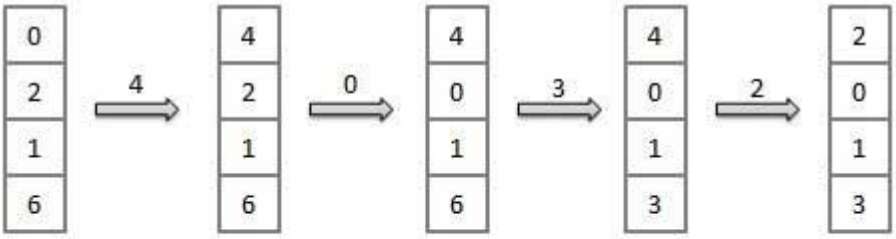
Least Recently Used (LRU) algorithm

- Page which has not been used for the longest time in main memory is the one which will be selected for replacement.

- Easy to implement, keep a list, replace pages by looking back into time.

Reference String : 0, 2, 1, 6, 4, 0, 1, 0, 3, 1, 2, 1

Misses : x x x x x x x x



Fault Rate = 8 / 12 = 0.67

Page Buffering algorithm

- To get a process start quickly, keep a pool of free frames.
- On page fault, select a page to be replaced.
- Write the new page in the frame of free pool, mark the page table and restart the process.
- Now write the dirty page out of disk and place the frame holding replaced page in free pool.

Least frequently Used (LFU) algorithm

- The page with the smallest count is the one which will be selected for replacement.
- This algorithm suffers from the situation in which a page is used heavily during the initial phase of a process, but then is never used again.

Most frequently Used (MFU) algorithm

- This algorithm is based on the argument that the page with the smallest count was probably just brought in and has yet to be used.

Memory extension

The physical address space on any computer system is directly limited by the number of address bits in an instruction. The physical address space may be extended by the MMU by providing additional address bits. Each memory address generated by the CPU passes through the MMU before arriving at the memory controller. The MMU attaches additional address bits which were previously specified by the OS. This expanded (or effective) address is passed to the memory controller, which retrieves the indicated word. In a PC, the physical address space is referred to as real memory and the logical address space is often referred to as virtual memory. When real memory is extended by hardware, the relationship between real and virtual memory can be changed in a significant manner. These relationships are shown below:

Physical (Real) Address space (R)

Logical (Virtual) address space (v)

Normal	**Extended**
V < R	V < R
V = R	V = R
V'> R* (V=R)	V > R

*Simulation of V > R; the instantaneous relationship is V = R at any instant.

The logical address space can be extended without hardware assistance by stimulating a larger memory through a technique known as **overlaying**.

Overlaying involves the replacement of a piece of the user's program by another piece-the overlay. Both program pieces share the same set of logical addresses. However, only one piece is assigned at any time during program execution. Overlaying is a method used to overcome main memory limitations. Overlaying allows a process to execute despite the system having insufficient physical memory. The programmer defines two or more overlay segments within the program such that no two overlay segments need be in memory at the same time. The OS would then manage swapping overlay segments, reducing the amount of physical memory required by the program. A major disadvantage of overlaying is the extensive programmer involvement in the process.

Dynamic Memory Allocation

Within the OS, a free pool of memory is usually set aside to accommodate the temporary needs of various system processes. This memory pool supports temporary system buffers, queue entries, and blocks for processes. As the OS usually operates in an unmapped mode (i.e., with the MMU disabled), this memory pool represents a contiguous segment that needs to be dynamically allocated to various requestors.

Many algorithms have been proposed for dynamic memory allocation. These are:

1. Best fit placement: This algorithm first searches the entire list of free partitions and considers the smallest hole that is adequate. The best fit deal with allocating the smallest free partition which meets the requirement of the requesting process. It then tries to find a hole which is close to actual process size needed. Whenever, a request is presented to the memory manager, it searches the list for the hole yielding the smallest DELTA as computed by:

HOLE_SIZE – NEEDED_SIZE → DELTA

Where, DELTA is the space remaining beyond the requested size.

Advantage: Memory utilization is much better as it searches the smallest free partition first available.

Disadvantage: It is slower and may even tend to fill up memory with tiny useless holes.

2. First-fit placement: This algorithm searches for the first hole or partition larger enough which can accommodate the process. It finishes after finding the first suitable free partition. Holes are maintained in the list by increasing address. The hole-list is searched for the first hole such that:

HOLE-SIZE >= NEEDED _SIZE

Advantage: Fastest algorithm because it searches as little as possible.

Disadvantage: The remaining unused memory areas left after allocation become waste if it is too smaller. Thus request for larger memory requirement cannot be accomplished.

3. Buddy system: The buddy system uses a power-of-2 algorithm to maintain its hole -lists. In buddy system, sizes of free blocks are in form of power of 2. For example, 2, 4, 8, 16, 32, 64, etc. Up to the size of memory. When a free block of size 2K is requested, a free block from the list of free blocks of size 2K is allocated. If no free block of size 2K is available, the block of next larger size, 2^{K+1} is split in two halves called buddies to satisfy the request.

Example: Let total memory size be 512KB and let a process P1, requires 70KB to be swapped in. As the hole lists are only for powers of 2, 128KB will be big enough. If no 128KB is there, nor are blocks of 256KB, a block of 512KB will be split into two halves giving 256KB each, one of them is further split into two 128KB blocks and one of the 128KB is allocated to the process.

Advantage: Buddy system is faster.

Disadvantage: It is often become inefficient in terms of memory utilization as all request must be rounded up to a power of 2.

4. Worst-Fit: In situations where best-fit finds a perfect match, the hole that remains is virtually useless because it is so small. To prevent the creation of these useless holes, worst fit works the opposite of best fit; it always picks the largest remaining hole.

Swapping: Swapping may be used to increase the number of processes sharing the CPU. In addition to the processes stored in the main memory, one or more processes may be temporarily copied out of memory to a

backing store. As space becomes available in main memory, swapped-out processes may be returned to memory. To execute on the CPU, a process must be in main memory. Decisions on which process to swap out and which process to swap in are made by the OS's process scheduler.

11.0 I/O Hardware

One of the important jobs of an Operating System is to manage various I/O devices including mouse, keyboards, touch pad, disk drives, display adapters, USB devices, Bit-mapped screen, LED, Analog-to-digital converter, On/off switch, network connections, audio I/O, printers etc.

An I/O system is required to take an application I/O request and send it to the physical device, then take whatever response comes back from the device and send it to the application. I/O devices can be divided into two categories −

- **Block devices** − A block device is one with which the driver communicates by sending entire blocks of data. For example, Hard disks, USB cameras, Disk-On-Key etc.
- **Character devices** − A character device is one with which the driver communicates by sending and receiving single characters (bytes, octets). For example, serial ports, parallel ports, sounds cards etc

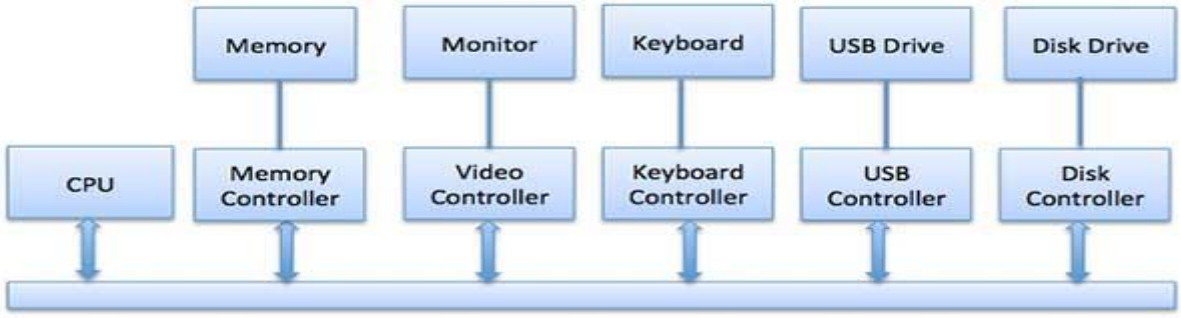
Device Controllers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices.

The Device Controller works like an interface between a device and a device driver. I/O units (Keyboard, mouse, printer, etc.) typically consist of a mechanical component and an electronic component where electronic component is called the device controller.

There is always a device controller and a device driver for each device to communicate with the Operating Systems. A device controller may be able to handle multiple devices. As an interface its main task is to convert serial bit stream to block of bytes, perform error correction as necessary.

Any device connected to the computer is connected by a plug and socket, and the socket is connected to a device controller. Following is a model for connecting the CPU, memory, controllers, and I/O devices where CPU and device controllers all use a common bus for communication.



Synchronous vs asynchronous I/O

- **Synchronous I/O** − In this scheme CPU execution waits while I/O proceeds
- **Asynchronous I/O** − I/O proceeds concurrently with CPU execution

Communication to I/O Devices

The CPU must have a way to pass information to and from an I/O device. There are three approaches available to communicate with the CPU and Device.

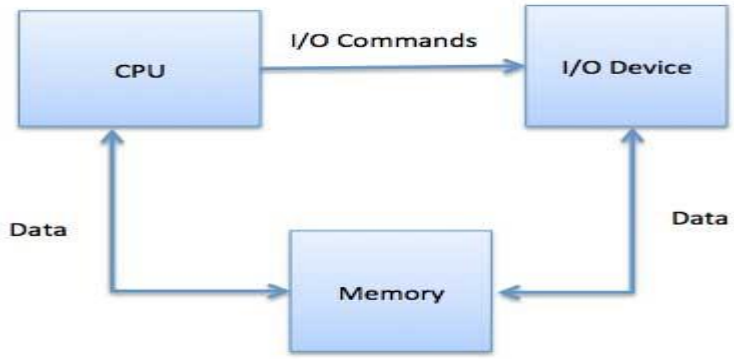
- Special Instruction I/O
- Memory-mapped I/O
- Direct memory access (DMA)

Special Instruction I/O

This uses CPU instructions that are specifically made for controlling I/O devices. These instructions typically allow data to be sent to an I/O device or read from an I/O device.

Memory-mapped I/O

When using memory-mapped I/O, the same address space is shared by memory and I/O devices. The device is connected directly to certain main memory locations so that I/O device can transfer block of data to/from memory without going through CPU.



While using memory mapped IO, OS allocates buffer in memory and informs I/O device to use that buffer to send data to the CPU. I/O device operates asynchronously with CPU, interrupts CPU when finished.

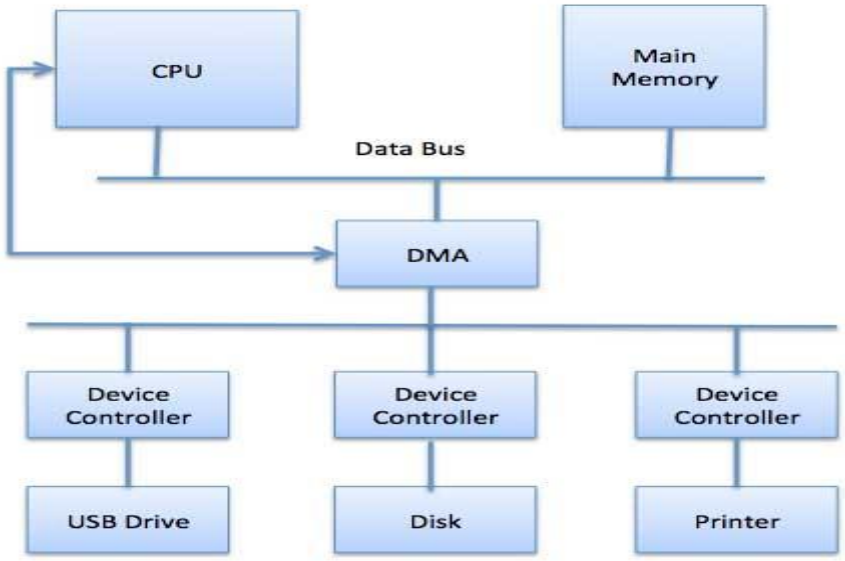
The advantage to this method is that every instruction which can access memory can be used to manipulate an I/O device. Memory mapped IO is used for most high-speed I/O devices like disks, communication interfaces.

Direct Memory Access (DMA)

Slow devices like keyboards will generate an interrupt to the main CPU after each byte is transferred. If a fast device such as a disk generated an interrupt for each byte, the operating system would spend most of its time handling these interrupts. So a typical computer uses direct memory access (DMA) hardware to reduce this overhead.

Direct Memory Access (DMA) means CPU grants I/O module authority to read from or write to memory without involvement. DMA module itself controls exchange of data between main memory and the I/O device. CPU is only involved at the beginning and end of the transfer and interrupted only after entire block has been transferred.

Direct Memory Access needs a special hardware called DMA controller (DMAC) that manages the data transfers and arbitrates access to the system bus. The controllers are programmed with source and destination pointers (where to read/write the data), counters to track the number of transferred bytes, and settings, which includes I/O and memory types, interrupts and states for the CPU cycles.



The operating system uses the DMA hardware as follows:

Step	Description
1	Device driver is instructed to transfer disk data to a buffer address X.
2	Device driver then instruct disk controller to transfer data to buffer.
3	Disk controller starts DMA transfer.
4	Disk controller sends each byte to DMA controller.
5	DMA controller transfers bytes to buffer, increases the memory address, decreases the counter C until C becomes zero.
6	When C becomes zero, DMA interrupts CPU to signal transfer completion.

Polling vs Interrupts I/O

A computer must have a way of detecting the arrival of any type of input. There are two ways that this can happen, known as **polling** and **interrupts**. Both of these techniques allow the processor to deal with events that can happen at any time and that are not related to the process it is currently running.

Polling I/O

Polling is the simplest way for an I/O device to communicate with the processor. The process of periodically checking status of the device to see if it is time for the next I/O operation, is called polling. The I/O device simply puts the information in a Status register, and the processor must come and get the information.

Most of the time, devices will not require attention and when one does it will have to wait until it is next interrogated by the polling program. This is an inefficient method and much of the processors time is wasted on unnecessary polls.

Compare this method to a teacher continually asking every student in a class, one after another, if they need help. Obviously the more efficient method would be for a student to inform the teacher whenever they require assistance.

Interrupts I/O

An alternative scheme for dealing with I/O is the interrupt-driven method. An interrupt is a signal to the microprocessor from a device that requires attention.

A device controller puts an interrupt signal on the bus when it needs CPU's attention when CPU receives an interrupt, It saves its current state and invokes the appropriate interrupt handler using the interrupt vector (addresses of OS routines to handle various events). When the interrupting device has been dealt with, the CPU continues with its original task as if it had never been interrupted.

Operating System - I/O Software

I/O software is often organized in the following layers:

- **User Level Libraries** − This provides simple interface to the user program to perform input and output. For example, **stdio** is a library provided by C and C++ programming languages.
- **Kernel Level Modules** − This provides device driver to interact with the device controller and device independent I/O modules used by the device drivers.
- **Hardware** − This layer includes actual hardware and hardware controller which interact with the device drivers and makes hardware alive.

A key concept in the design of I/O software is that it should be device independent where it should be possible to write programs that can access any I/O device without having to specify the device in advance. For example, a program that reads a file as input should be able to read a file on a floppy disk, on a hard disk, or on a CD-ROM, without having to modify the program for each different device.



Device Drivers

Device drivers are software modules that can be plugged into an OS to handle a particular device. Operating System takes help from device drivers to handle all I/O devices. Device drivers encapsulate device-dependent code and implement a standard interface in such a way that code contains device-specific register reads/writes. Device driver, is generally written by the device's manufacturer and delivered along with the device on a CD-ROM.

A device driver performs the following jobs:

- To accept request from the device independent software above to it.
- Interact with the device controller to take and give I/O and perform required error handling
- Making sure that the request is executed successfully

How a device driver handles a request is as follows:

Suppose a request comes to read a block N. If the driver is idle at the time a request arrives, it starts carrying out the request immediately. Otherwise, if the driver is already busy with some other request, it places the new request in the queue of pending requests.

Interrupt handlers

An interrupt handler, also known as an interrupt service routine or ISR, is a piece of software or more specifically a callback function in an operating system or more specifically in a device driver, whose execution is triggered by the reception of an interrupt.

When the interrupt happens, the interrupt procedure does whatever it has to in order to handle the interrupt, updates data structures and wakes up process that was waiting for an interrupt to happen.

The interrupt mechanism accepts an address, which is a number that selects a specific interrupt handling routine/function from a small set. In most architectures, this address is an offset stored in a table called the interrupt vector table. This vector contains the memory addresses of specialized interrupt handlers.

Device-Independent I/O Software

The basic function of the device-independent software is to perform the I/O functions that are common to all devices and to provide a uniform interface to the user-level software. Though it is difficult to write completely device independent software but we can write some modules which are common among all the devices. Following is a list of functions of device-independent I/O Software −

- Uniform interfacing for device drivers
- Device naming - Mnemonic names mapped to Major and Minor device numbers
- Device protection
- Providing a device-independent block size
- Buffering because data coming off a device cannot be stored in final destination.
- Storage allocation on block devices
- Allocation and releasing dedicated devices
- Error Reporting

User-Space I/O Software

These are the libraries which provide richer and simplified interface to access the functionality of the kernel or ultimately interactive with the device drivers. Most of the user-level I/O software consists of library procedures with some exception like spooling system which is a way of dealing with dedicated I/O devices in a multiprogramming system.

I/O Libraries (e.g., stdio) are in user-space to provide an interface to the OS resident device-independent I/O SW. For example putchar(), getchar(), printf() and scanf() are example of user level I/O library stdio available in C programming.

Kernel I/O Subsystem

Kernel I/O Subsystem is responsible to provide many services related to I/O. Following are some of the services provided.

- **Scheduling** − Kernel schedules a set of I/O requests to determine a good order in which to execute them. When an application issues a blocking I/O system call, the request is placed on the queue for that device. The Kernel I/O scheduler rearranges the order of the queue to improve the overall system efficiency and the average response time experienced by the applications.
- **Buffering** − Kernel I/O Subsystem maintains a memory area known as **buffer** that stores data while they are transferred between two devices or between a device with an application operation. Buffering is done to cope with a speed mismatch between the producer and consumer of a data stream or to adapt between devices that have different data transfer sizes.
- **Caching** − Kernel maintains cache memory which is region of fast memory that holds copies of data. Access to the cached copy is more efficient than access to the original.
- **Spooling and Device Reservation** − A spool is a buffer that holds output for a device, such as a printer, that cannot accept interleaved data streams. The spooling system copies the queued spool files to the printer one at a time. In some operating systems, spooling is managed by a system daemon process. In other operating systems, it is handled by an in kernel thread.
- **Error Handling** − An operating system that uses protected memory can guard against many kinds of hardware and application errors.

Memory Caching: - is the most popular and usually the most effective scheme for improving memory performance. This technique relies on a small amount (8k to 512k) of raw, high speed memory fast enough to keep up with the processor with zero wait states. This small bank of cache memory often is rated at 15ns or less in access speed. Because this rate is faster than normal DRAM components can handle, a special type of memory is used. This memory is called SRAM. SRAM devices do not need the constant refresh signals that DRAM devices require. Although SRAM chips are expensive only a small number of SRAM chips are required in a caching scheme. SRAM is used by a special cache-controller circuit that stores frequently accessed RAM locations and also is reloaded with the RAM values that the cache controller

Effective Access Time Calculation

We can calculate the estimated cost of page faults by performing an *effective access time* calculation. The basic idea is that sometimes you access a location quickly (there is no page fault) and sometimes more slowly (you have to wait for a page to come into memory). We use the cost of each type of access and the percentage of time that it occurs to computer the average time to access a word of memory.

Let:

- h = fraction of time that a reference does **not** require a page fault.
- t_{mem} = time it takes to read a word from memory.
- t_{disk} = time it takes to read a page from disk.

then

- $EAT = h * t_{mem} + (1 - h) * t_{disk}$.

If there are multiple classes of memory accesses, such as no disk access, one disk access, and two disk access, then you would have a fraction (h) and access time (t) for each class of access.

Note that this calculation is the same type that computer architects use to calculate memory performance. In that case, their access classes might be (1) cached in L1, (2) cached in L2, and (3) RAM.

12. The Concept of Cache Hit versus Cache Miss

A cache memory requires a cache controller to determine whether or not the data currently being accessed by the CPU resides in the cache or whether it must be obtained from the main memory. When the current address is applied to the cache controller, the controller returns a signal called **hit**, which is asserted if the data is currently in the cache. A **cache hit** means that the particular data the CPU wanted was available in the cache RAM and no additional wait states are needed for retrieving this data.

A **cache miss** – means that the data the CPU wanted had not been loaded into the cache RAM and that wait states must be inserted while the data is retrieved. A good cache controller has a hit ratio of 95 percent or more. The principal parameter of a cache system is its hit ratio, H, which determines the ratio of hits to all accesses. The following parameters are readily used:

Let;

Access time of main memory be tm,

Access time of cache memory be tc,

Hit ratio be H,

Miss ratio be M,

Speed-up ratio be S.

The speed-up ratio is defined as the ratio of the memory system's access time without cache to its speed with cache. For N accesses to memory the total access time of a memory without cache is given by: Ntm. For N accesses to a memory with cache the total access time is given by: N (Htc + Mtm).

The speed-up ratio is therefore given by S = Ntm/N(Htc + Mtm) =tm/ (Htc + Mtm).

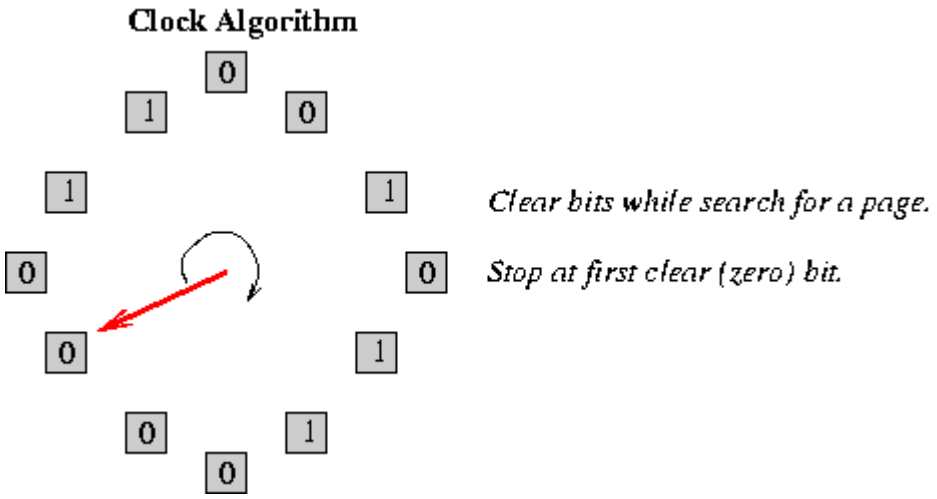
If M is expressed in terms of H as M = 1- H; then by substitution:

S = tm/ (Htc + (1 – H)tm).

THRASHING: CLOCK ALGORITHM, THRASHING

This is an efficient way to approximate LRU.

Clock algorithm: keeps "use" bit for each page frame, hardware sets the appropriate bit on every memory reference. The operating system clears the bits from time to time in order to figure out how often pages are being referenced. The OS uses clock algorithm to find a page where to throw out physical frames, clearing use bits until one is found that is zero and uses that one. Figure below shows clock analogy.



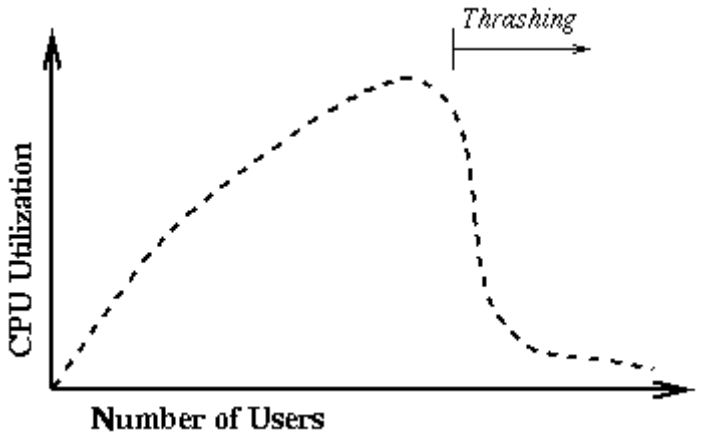
Fancier algorithm: give pages a second (third? fourth?) chance. Store (in software) a counter for each page frame, and increment the counter if use bit is zero. Only throw the page out if the counter passes a certain limit value. Limit = 0 corresponds to the previous case. What happens when limit is small? large?

Some systems also use a "dirty" bit to give preference to dirty pages. This is because it is more expensive to throw out dirty pages: clean ones need not be written to disk.

Thrashing: consider what happens when memory gets overcommitted.

- Suppose there are many users, and that between them their processes are making frequent references to 50 pages, but memory has 40 pages.
- Each time one page is brought in, another page, whose contents will soon be referenced, is thrown out.
- Compute average memory access time.
- The system will spend all of its time reading and writing pages. It will be working very hard but not getting anything done.
- Thrashing was a severe problem in early demand paging systems.

Thrashing occurs because the system does not know when it has taken on more work than it can handle. LRU mechanisms order pages in terms of last access, but do not give absolute numbers indicating pages that *must not* be thrown out.



What can be done?

- If a single process is too large for memory, there is nothing the OS can do. That process will simply thrash.
- If the problem arises because of the sum of several processes:
    - Figure out how much memory each process needs.
    - Change scheduling priorities to run processes in groups whose memory needs can be satisfied.

Thrashing is a condition that can befall systems managed by the paging technique. It is denoted by excessive overhead In the OS and severe performance degradation in the overall system throughput.

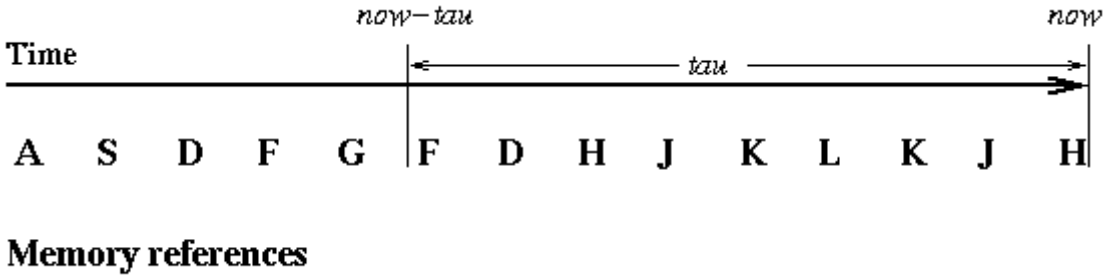
Thrashing is the result of a shortage in memory space, which produces a surplus of processor cycles. The concept of thrashing relies on the fact that the system goes from a period of steady- state efficiency into chaotic movement of pages between storage and main memory.

Degraded performance because of excessive swapping is known as thrashing.

Working Sets

Working Sets are a solution proposed by Peter Denning. An informal definition is "the collection of pages that a process is working with, and which must thus be resident if the process is to avoid thrashing." The idea is to use the recent needs of a process to predict its future needs.

- Choose tau, the working set parameter. At any given time, all pages referenced by a process in its last tau seconds of execution are considered to comprise its *working set*.



- A process will never be executed unless its working set is resident in main memory. Pages outside the working set may be discarded at any time.

Working sets are not enough by themselves to make sure memory does not get overcommitted. We must also introduce the idea of a *balance set*:

- If the sum of the working sets of all runnable processes is greater than the size of memory, then refuse to run some of the processes (for a while).
- Divide runnable processes up into two groups: active and inactive. When a process is made active its working set is loaded, when it is made inactive its working set is allowed to migrate back to disk. The collection of active processes is called the *balance set.*
- Some algorithm must be provided for moving processes into and out of the balance set. What happens if the balance set changes too frequently?

As working sets change, corresponding changes will have to be made in the balance set.

Problem with the working set: must constantly be updating working set information.

- One of the initial plans was to store some sort of a capacitor with each memory page. The capacitor would be charged on each reference, then would discharge slowly if the page was not referenced. Tau would be determined by the size of the capacitor. This was not actually implemented. One problem is that we want separate working sets for each process, so the capacitor should only be allowed to discharge when a particular process executes. What if a page is shared?
- Actual solution: take advantage of use bits

- OS maintains *idle time* value for each page: amount of CPU time received by process since last access to page.
- Every once in a while, scan all pages of a process. For each use bit on, clear page's idle time. For use bit off, add process' CPU time (since last scan) to idle time. Turn all use bits off during scan.
- Scans happen on order of every few seconds (in Unix, tau is on the order of a minute or more).

This actual solution is an approximation algorithm known as *WSClock*.

The working set w(t) is the smallest collection of pages of a given program that, resident in memory (i.e., currently executing process). A working set principle can be stated as follows: a program may execute if and only if its working set pages are currently resident in memory.

Furthermore, a page may not be removed from memory if it is the member of a working set of a running program. The working set exhibits several characteristics:

Suppose that w(h) is the expected working set size, that is,

W(h)= expected value [w(t,h)].

where h – is the number of pages; Then the following properties has been identified:

(1) Given $h \geq 1$, then $1 \leq w(h) < \min(n,h)$. That is a program's working set is always one and can be no bigger than the window size or total number of pages in the program.

(2) Then $w(h) \leq w(h+i)$. That is, the working set increases up to the limit over time.

13.0 OS File Systems

What is a File?

A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks. In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

File Structure

A File Structure should be according to a required format that the operating system can understand.

- A file has a certain defined structure according to its type.
- A text file is a sequence of characters organized into lines.
- A source file is a sequence of procedures and functions.
- An object file is a sequence of bytes organized into blocks that are understandable by the machine.
- When operating system defines different file structures, it also contains the code to support these file structure. Unix, MS-DOS support minimum number of file structure.

File Type

File type refers to the ability of the operating system to distinguish different types of file such as text files source files and binary files etc. Many operating systems support many types of files. Operating system like MS-DOS and UNIX have the following types of files −

Ordinary files

- These are the files that contain user information.
- These may have text, databases or executable program.

- The user can apply various operations on such files like add, modify, delete or even remove the entire file.

Directory files

- These files contain list of file names and other information related to these files.

Special files

- These files are also known as device files.
- These files represent physical device like disks, terminals, printers, networks, tape drive etc.

These files are of two types −

- **Character special files** − data is handled character by character as in case of terminals or printers.
- **Block special files** − data is handled in blocks as in the case of disks and tapes.

File Access Mechanisms

File access mechanism refers to the manner in which the records of a file may be accessed. There are several ways to access files −

- Sequential access
- Direct/Random access
- Indexed sequential access

Sequential access

A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other. This access method is the most primitive one. Example: Compilers usually access files in this fashion.

Direct/Random access

- Random access file organization provides, accessing the records directly.
- Each record has its own address on the file with by the help of which it can be directly accessed for reading or writing.
- The records need not be in any sequence within the file and they need not be in adjacent locations on the storage medium.

Indexed sequential access

- This mechanism is built up on base of sequential access.
- An index is created for each file which contains pointers to various blocks.
- Index is searched sequentially and its pointer is used to access the file directly.

Space Allocation

Files are allocated disk spaces by operating system. Operating systems deploy following three main ways to allocate disk space to files.

- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

Contiguous Allocation

- Each file occupies a contiguous address space on disk.
- Assigned disk address is in linear order.
- Easy to implement.
- External fragmentation is a major issue with this type of allocation technique.

Linked Allocation

- Each file carries a list of links to disk blocks.
- Directory contains link / pointer to first block of a file.
- No external fragmentation
- Effectively used in sequential access file.
- Inefficient in case of direct access file.

Indexed Allocation

- Provides solutions to problems of contiguous and linked allocation.
- A index block is created having all pointers to files.
- Each file has its own index block which stores the addresses of disk space occupied by the file.
- Directory contains the addresses of index blocks of files.

14.0 OS Security

Security refers to providing a protection system to computer system resources such as CPU, memory, disk, software programs and most importantly data/information stored in the computer system. If a computer program is run by an unauthorized user, then he/she may cause severe damage to computer or data stored in it. So a computer system must be protected against unauthorized access, malicious access to system memory, viruses, worms etc. We're going to discuss the following topics in this chapter.

- Authentication
- One Time passwords
- Program Threats
- System Threats
- Computer Security Classifications

Authentication

Authentication refers to identifying each user of the system and associating the executing programs with those users. It is the responsibility of the Operating System to create a protection system which ensures that a user who is running a particular program is authentic. Operating Systems generally identifies/authenticates users using following three ways:

- **Username/Password** − User need to enter a registered username and password with Operating system to login into the system.
- **User card/key** − User need to punch card in card slot, or enter key generated by key generator in option provided by operating system to login into the system.
- **User attribute - fingerprint/ eye retina pattern/ signature** − User need to pass his/her attribute via designated input device used by operating system to login into the system.

One Time passwords

One-time passwords (OTPs) provide additional security along with normal authentication. In One-Time Password system, a unique password is required every time a user tries to login into the system. Once a one-time password is used, then it cannot be used again. One-time password are implemented in various ways.

- **Random numbers** − Users are provided cards having numbers printed along with corresponding alphabets. System asks for numbers corresponding to few alphabets randomly chosen.
- **Secret key** − User are provided a hardware device which can create a secret id mapped with user id. System asks for such secret id which is to be generated every time prior to login.
- **Network password** − Some commercial applications send one-time passwords to user on registered mobile/ email which is required to be entered prior to login.

Program Threats

Operating system's processes and kernel do the designated task as instructed. If a user program made these process do malicious tasks, then it is known as **Program Threats**. One of the common example of program threat is a program installed in a computer which can store and send user credentials via network to some hacker. Following is the list of some well-known program threats.

- **Trojan Horse** − Such program traps user login credentials and stores them to send to malicious user who can later on login to computer and can access system resources.
- **Trap Door** − If a program which is designed to work as required, have a security hole in its code and perform illegal action without knowledge of user then it is called to have a trap door.
- **Logic Bomb** − Logic bomb is a situation when a program misbehaves only when certain conditions met otherwise it works as a genuine program. It is harder to detect.
- **Virus** − Virus as name suggest can replicate themselves on computer system. They are highly dangerous and can modify/delete user files, crash systems. A virus is generatlly a small code embedded in a program. As user accesses the program, the virus starts getting embedded in other files/ programs and can make system unusable for user

System Threats

System threats refers to misuse of system services and network connections to put user in trouble. System threats can be used to launch program threats on a complete network called as program attack. System

threats creates such an environment that operating system resources/ user files are misused. Following is the list of some well-known system threats.

- **Worm** − Worm is a process which can choked down a system performance by using system resources to extreme levels. A Worm process generates its multiple copies where each copy uses system resources, prevents all other processes to get required resources. Worms processes can even shut down an entire network.
- **Port Scanning** − Port scanning is a mechanism or means by which a hacker can detects system vulnerabilities to make an attack on the system.
- **Denial of Service** − Denial of service attacks normally prevents user to make legitimate use of the system. For example, a user may not be able to use internet if denial of service attacks browser's content settings.

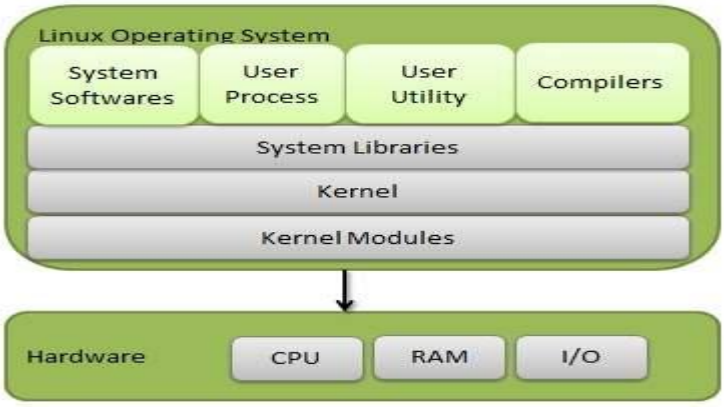
14.0 Linux Operating System

Linux is one of popular version of UNIX operating System. It is open source as its source code is freely available. It is free to use. Linux was designed considering UNIX compatibility. Its functionality list is quite similar to that of UNIX.

Components of Linux System

Linux Operating System has primarily three components

- **Kernel** − Kernel is the core part of Linux. It is responsible for all major activities of this operating system. It consists of various modules and it interacts directly with the underlying hardware. Kernel provides the required abstraction to hide low level hardware details to system or application programs.
- **System Library** − System libraries are special functions or programs using which application programs or system utilities accesses Kernel's features. These libraries implement most of the functionalities of the operating system and do not requires kernel module's code access rights.
- **System Utility** − System Utility programs are responsible to do specialized, individual level tasks.



Kernel Mode vs User Mode

Kernel component code executes in a special privileged mode called **kernel mode** with full access to all resources of the computer. This code represents a single process, executes in single address space and do not require any context switch and hence is very efficient and fast. Kernel runs each processes and provides system services to processes, provides protected access to hardware to processes.

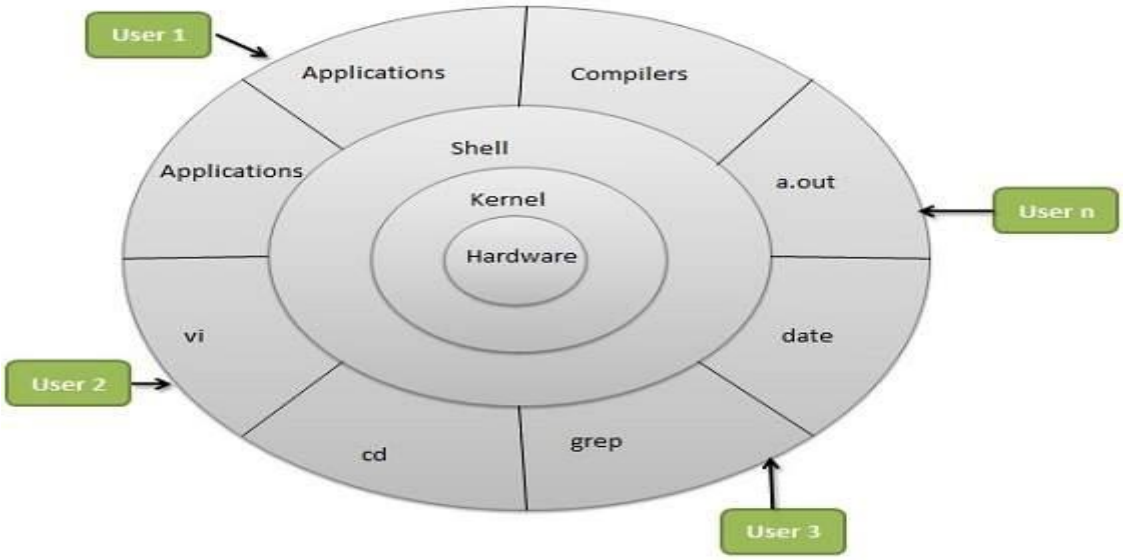
Support code which is not required to run in kernel mode is in System Library. User programs and other system programs works in **User Mode** which has no access to system hardware and kernel code. User programs/ utilities use System libraries to access Kernel functions to get system's low level tasks.

Basic Features

Following are some of the important features of Linux Operating System.

- **Portable** − Portability means software can works on different types of hardware in same way. Linux kernel and application programs supports their installation on any kind of hardware platform.
- **Open Source** − Linux source code is freely available and it is community based development project. Multiple teams work in collaboration to enhance the capability of Linux operating system and it is continuously evolving.
- **Multi-User** − Linux is a multiuser system means multiple users can access system resources like memory/ ram/ application programs at same time.
- **Multiprogramming** − Linux is a multiprogramming system means multiple applications can run at same time.
- **Hierarchical File System** − Linux provides a standard file structure in which system files/ user files are arranged.
- **Shell** − Linux provides a special interpreter program which can be used to execute commands of the operating system. It can be used to do various types of operations, call application programs. etc.
- **Security** − Linux provides user security using authentication features like password protection/ controlled access to specific files/ encryption of data.

Architecture: The following illustration shows the architecture of a Linux system:



The architecture of a Linux System consists of the following layers −

- **Hardware layer** − Hardware consists of all peripheral devices (RAM/ HDD/ CPU etc).
- **Kernel** − It is the core component of Operating System, interacts directly with hardware, provides low level services to upper layer components.
- **Shell** − An interface to kernel, hiding complexity of kernel's functions from users. The shell takes commands from the user and executes kernel's functions.
- **Utilities** − Utility programs that provide the user most of the functionalities of an operating systems.

Linux File System Structure

Linux file system has a hierarchal file structure as it contains a root directory and its subdirectories. All other directories can be accessed from the root directory. A partition usually has only one file system, but it may have more than one file system.

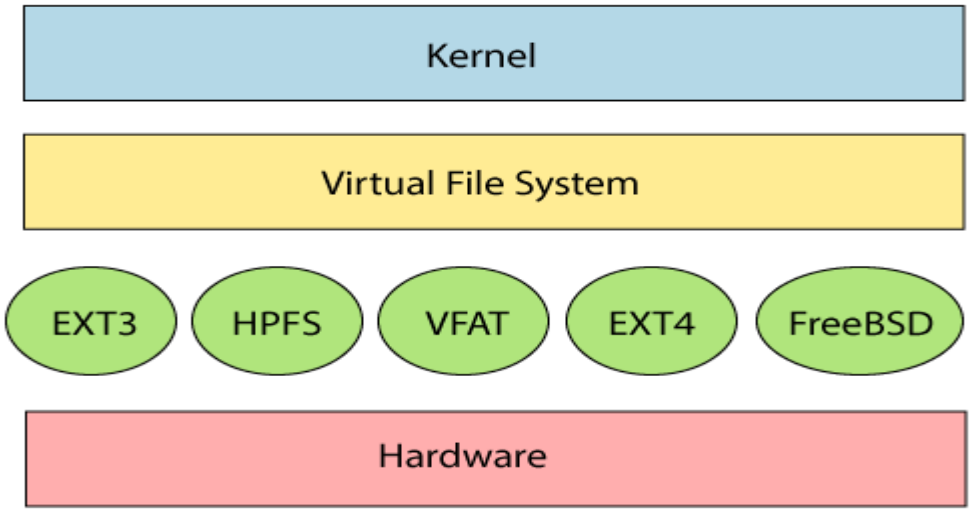
A file system is designed in a way so that it can manage and provide space for non-volatile storage data. All file systems required a namespace that is a naming and organizational methodology. The namespace defines the naming process, length of the file name, or a subset of characters that can be used for the file name. It also defines the logical structure of files on a memory segment, such as the use of directories for organizing the specific files. Once a namespace is described, a Metadata description must be defined for that particular file.

The data structure needs to support a hierarchical directory structure; this structure is used to describe the available and used disk space for a particular block. It also has the other details about the files such as file size, date & time of creation, update, and last modified.

Also, it stores advanced information about the section of the disk, such as partitions and volumes.

The advanced data and the structures that it represents contain the information about the file system stored on the drive; it is distinct and independent of the file system metadata.

Linux file system contains two-part file system software implementation architecture. Consider the figure 24 below:



Acronyms: HPFS: High performance file system, VFAT: Virtual file Allocation table, FreeBSD: Free Berkeley Software Distribution (an open-Source Unix –like OS)

Linux file system

The Linux file system architecture consists of the following components:

Kernel: Linux kernel is the main component of a Linux OS and is the core interface between a computer's hardware and its processes. It communicates between the hardware and processes, managing resources as efficiently as possible. The kernel has four jobs: (i) Memory management, (ii) process management, (iii) Device management and (iv) System calls and security

Virtual File System (VFS): The VFS acts as the interface between the user and the file's backing filesystem, masking any implementation details behind generic calls such as open(), read(), write(), etc. It offers many file system types such as **Ext, Ext2, Ext3, Ext4, JFS, ReiserFS, XFS, btrfs,** and **swap**.

Hardware: This refers to all set of hardware of the given computer. Linux installation depends on the hardware requirements of the computer.

The file system requires an API (Application programming interface) to access the function calls to interact with file system components like files and directories. API facilitates tasks such as creating, deleting, and copying the files. It facilitates an algorithm that defines the arrangement of files on a file system.

The first two parts of the given file system together called a **Linux virtual file system**. It provides a single set of commands for the kernel and developers to access the file system. This virtual file system requires the specific system driver to give an interface to the file system.

Linux File System Features

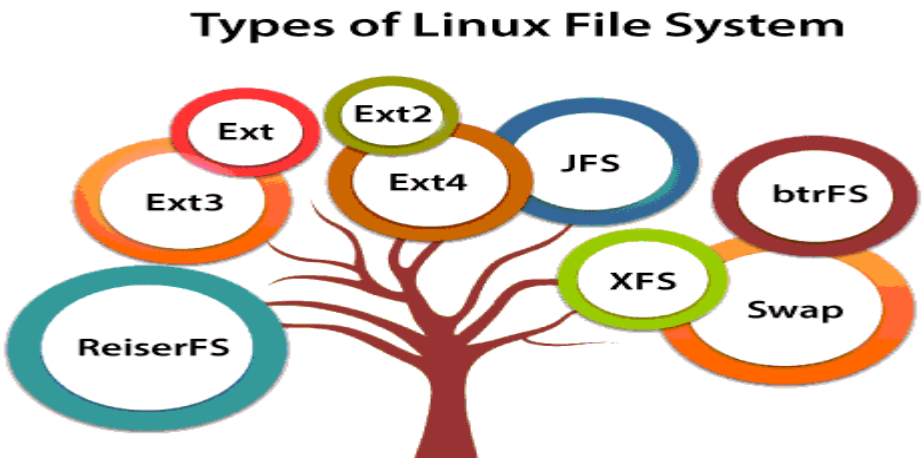
In Linux, the file system creates a tree structure. All the files are arranged as a tree and its branches. The topmost directory called the **root (/) directory**. All other directories in Linux can be accessed from the root directory.

Some key features of Linux file system are as following:

- **Specifying paths:** Linux does not use the backslash (\) to separate the components; it uses forward slash (/) as an alternative. For example, as in Windows, the data may be stored in C:\ My Documents\ Work, whereas, in Linux, it would be stored in /home/ My Document/ Work.
- **Partition, Directories, and Drives:** Linux does not use drive letters to organize the drive as Windows does. In Linux, we cannot tell whether we are addressing a partition, a network device, or an "ordinary" directory and a Drive.
- **Case Sensitivity:** Linux file system is case sensitive. It distinguishes between lowercase and uppercase file names. Such as, there is a difference between test.txt and Test.txt in Linux. This rule is also applied for directories and Linux commands.
- **File Extensions:** In Linux, a file may have the extension '.txt,' but it is not necessary that a file should have a file extension. While working with Shell, it creates some problems for the beginners to differentiate between files and directories. If we use the graphical file manager, it symbolizes the files and folders.
- **Hidden files:** Linux distinguishes between standard files and hidden files, mostly the configuration files are hidden in Linux OS. Usually, we don't need to access or read the hidden files. The hidden files in Linux are represented by a dot (.) before the file name (e.g., .ignore). To access the files, we need to change the view in the file manager or need to use a specific command in the shell.

Types of Linux File System

When we install the Linux operating system, Linux offers many file systems such as **Ext, Ext2, Ext3, Ext4, JFS, ReiserFS, XFS, btrfs,** and **swap**.



Types of Linux file system

Let's understand each of these file systems in detail:

1. Ext, Ext2, Ext3 and Ext4 file system

The file system Ext stands for **Extended File System**. It was primarily developed for **MINIX OS**. The Ext file system is an older version, and is no longer used due to some limitations.

Ext2 is the first Linux file system that allows managing two terabytes of data. Ext3 is developed through Ext2; it is an upgraded version of Ext2 and contains backward compatibility. The major drawback of Ext3 is that it does not support servers because this file system does not support file recovery and disk snapshot.

Ext4 file system is the faster file system among all the Ext file systems. It is a very compatible option for the SSD (solid-state drive) disks, and it is the default file system in Linux distribution.

2. JFS File System

JFS stands for **Journaled File System**, and it is developed by **IBM for AIX Unix**. It is an alternative to the Ext file system. It can also be used in place of Ext4, where stability is needed with few resources. It is a handy file system when CPU power is limited.

3. ReiserFS File System

ReiserFS is an alternative to the Ext3 file system. It has improved performance and advanced features. In the earlier time, the ReiserFS was used as the default file system in SUSE Linux, but later it has changed some policies, so SUSE returned to Ext3. This file system dynamically supports the file extension, but it has some drawbacks in performance.

4. XFS File System

XFS file system was considered as high-speed JFS, which is developed for parallel I/O processing. NASA still using this file system with its high storage server (300+ Terabyte server).

5. Btrfs File System

Btrfs stands for the **B tree file system**. It is used for fault tolerance, repair system, fun administration, extensive storage configuration, and more. It is not a good suit for the production system.

6. Swap File System

The swap file system is used for memory paging in Linux operating system during the system hibernation. A system that never goes in hibernate state is required to have swap space equal to its RAM size.

11.0 Windows (NT) File System

The Windows file system is called NTFS, and was introduced with Windows NT 4.0 and is the standard file system on Windows 2000 and later systems, such as Windows XP. Its goal was to solve the size, performance, reliability, and flexibility limitations in the DOS (aka "FAT" file system).

It has a general similarity to the FAT file system in that all files are described in a single table, called the *Master File Table* (MFT). However, it has more modern characteristics in that all components are files, including:

- Master File Table
- data files
- directories
- free list (bit map)
- boot images
- recovery logs

The file system also has features to support redundancy and transactions, which we will not discuss. A great reference for details is the book: **Inside the Windows NT File System** by Helen Custer, published by (not surprisingly) Microsoft Press.

Disk Layout

Disks are divided into fixed size regions:

- Each region is called a *volume*.
- Each volume can contain a different kind of file system, such as NTFS, FAT, or even Unix.
- Since each volume is a separate file system, it has its own root directory.
- Multiple volumes allow for fixed limits on the growth of a particular file tree, such as limiting the size of temporary file space.
- Multiple volumes also allow a single disk to contain multiple, separating bootable operating system.

Master File Table (MFT)

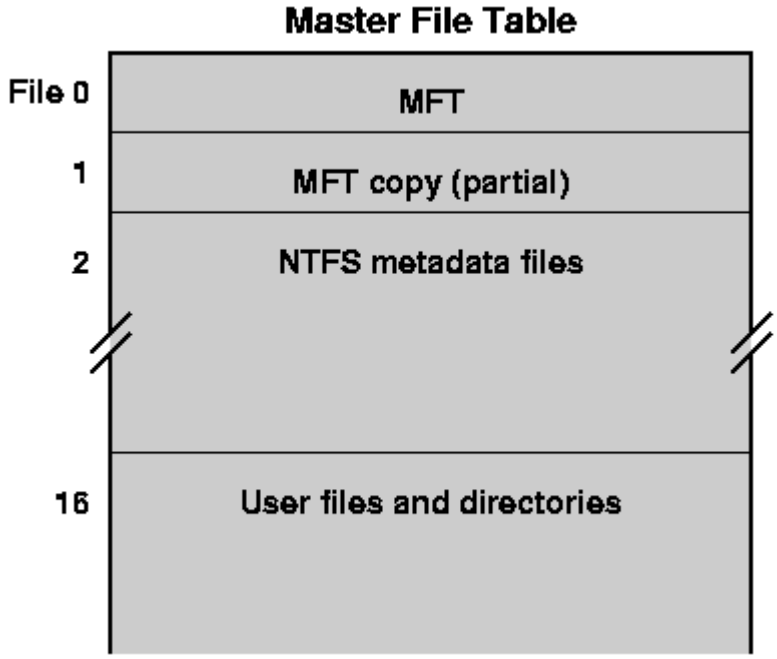
Clusters are the key element to allocation:

- Logically, the disk consists of allocation units called ***clusters***.
- A cluster is a power-of-two multiple of the physical disk block size. The cluster size is set when the disk is formatted. A small cluster provides a finer granularity of allocation, but may require more space to describe the file and more separate operations to transfer data to or from memory.
- The free list is a **bitmap**, each of whose bits describe one cluster.

- Clusters on the disk are numbered starting from zero to the maximum number of clusters (minus one). These numbers are called *logical cluster numbers* (LCN) and are used to name blocks (clusters) on disk.

The MFT is the major, and in some ways, the only data structure on disk:

- All files, and therefore all objects stored on disk are described by the MFT.
- All files are logical stored *in* the MFT and, for small files are physically within the bounds of the MFT. In this sense, the MFT *is* the file system.
- The MFT logically can be described as a table with one row per file.
- The first rows in the table described important configuration files, including for the MFT itself.



MFT Entries

As stated previously, each row or entry in the MFT (called a *record*) describes a file and logically contains the file. In the case of small files, the entry actually contains the contents of the file.

Each entry is consists of (attribute, value) pairs. While the conceptual design of NTFS is such that this set of pairs is extensible to include user-defined attributes, current version of NTFS have a fixed set. The main attributes are:

- *Standard information:* This attribute includes the information that was standard in the MS-DOS world:
  - read/write permissions,
  - creation time,
  - last modification time,
  - count of how many directories point to this this file (hard link count.
- *File Name:* This attribute describes the file's name in the Unicode character set. Multiple file names are possible, such as when:
  - the file has multiple links, or
  - the file has an MS-DOS short name.
- *Security Descriptor:* This attribute lists which user owns the file and which users can access it (and how they can access it).

- *Data:* This attribute either contains the actual file data in the case of a small file or points to the data (or points to the objects that point to the data) in the case of larger files.

MFT Entry (Simplified)

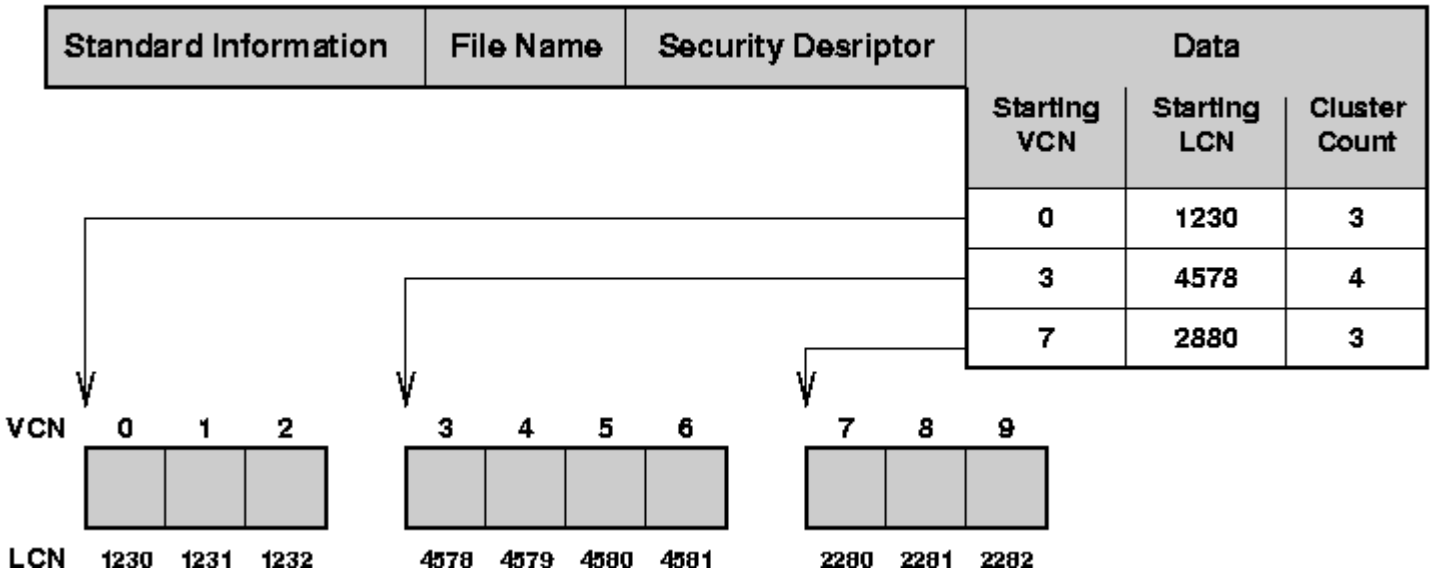
Standard Information	File Name	Security Desriptor	Data

For small files, this design is extremely efficient. By looking no further than the MFT entry, you have the complete contents of the file.

However, the Data field gets interesting when the data contained in the file is larger than an MFT entry. When dealing with large data, the Data attribute contains *pointers* to the data, rather than the data itself.

- The pointers to data are actually pointers to sequences of logical clusters on the disk.
- Each sequence is identified by three parts:
  - starting cluster in the file, called the *virtual cluster number* (VCN),
  - starting logical cluster (LCN) of the sequence on disk,
  - length, counted as the number of clusters.
- The run of clusters is called an *extent*, following the terminology developed by IBM in the 1960's.
- NTFS allocates new extents as necessary. When there is no more space left in the MFT entry, then another MFT entry is allocated. This design is effectively a list of extents, rather than the Unix or DEMOS tree of extents.

MFT Entry (with extents)



Directories

As with other modern file systems, a directory in NTFS is a file whose data contains a collection of name/file mappings.

- A directory entry contains the name of the file and *file reference*. The file references indentifies the file on this volume. In other words, it is an internal name for the file. A reference is a (*file number*, sequence number) pair. The file number is the offset of the file's entry in the MFT table. It is similar to the Unix inumber (Inode number).
- The list of file names in the directories is not stored in a simple list, but rather as a lexigraphically-sorted tree, called a B+ tree (this will be familiar to those with a database background). The data structure is called an *index* in NFTS (again, following the terminology from databases).
- The NTFS design specifies that an index can be constructed for any attribute, but currently only file name indices are supported.
- The name for a file appears both in its directory entry and in the MFT entry for the file itself.
- As with regular files, if the directory is small enough, it can fit entirely within the MFT entry.

MFT Directory Entry (Everything Fits)



If the directory is larger, then the top part of (the B+ tree of) the directory is in the MFT entry, which points to extents that contain the rest of the name/file mappings.

15.0 Anatomy of a Mobile Operating System

What is a mobile operating system?

A mobile operating system is software that allows smartphones, tablets and other devices to run applications and programs. A mobile OS provides an interface between the device's hardware components and its software functions. It typically starts when a device powers on, presenting a screen with icons or tiles that show information and provide application access. Mobile operating systems also manage cellular and wireless network connectivity and phone access.

Millions of people use mobile operating systems worldwide, powering a wide range of devices, from smartphones to tablets and wearable technology. These systems offer users a wide selection of features, including calling and messaging, internet and cellular data connectivity, multitasking capabilities, interactive user interfaces and access to a wide range of third-party applications and services to enhance the user experience even further.

Types of mobile operating systems

There are several mobile operating systems on the market today, but two of the most widely adopted are the iPhone's OS, Apple iOS, and Google's open source OS, Google Android. These two mobile OSes take different approaches to the mobile operating system and end-user experience.

Apple distributes the only devices that natively support iOS, and it takes a "walled garden" approach, in which Apple regulates all mobile apps and services that can run on iOS devices. Apple developed iOS to run on its own XNU kernel. Apple has also released several device-specific mobile operating systems, such as watchOS for the Apple Watch and iPadOS for iPad tablets.

Google takes an Open Source approach with Android, which allows mobile device OEMs (original equipment manufacturers) to customize the Android source code to fit their devices. Android runs on the Linux kernel. Additionally, due to being open source and globally popular, Android has become a staple in development for various equipment such as kiosks, machinery, TV operating systems and more.

Other mobile operating systems are available, but their adoption rates are well below those of iOS and Android and are mostly found outside of the U.S. These include operating systems like KaiOS, Sailfish OS and Huawei's HarmonyOS.

Several mobile operating systems are no longer commonly supported. BlackBerry 10, Nokia's Symbian, LG webOS -- formerly Palm webOS and HP webOS -- and Microsoft's Windows Phone OS are all no longer supported.

Features of mobile operating systems

Mobile operating systems deliver various features to users. The distinguishing feature that mobile operating systems offer is the ability to connect to the internet via the smartphone's built-in modem and a wireless service provider such as Verizon or AT&T. This is a major difference between mobile operating systems and most desktop OSes, which rely on a Wi-Fi network or Ethernet connection to access the internet, giving end users the ability to work remotely.

Mobile OSes offer a native web browser application, which allows users to search the internet and visit webpages. Mobile operating systems also provide application stores, which enable users to download and interface with mobile applications. Several mobile operating systems also have native GPS (global positioning system) applications that allow users to search for locations, follow step-by-step directions and, in some cases, share locations with different devices. The GPS feature relies on the mobile device's hardware and can't run without that support.

Other common mobile operating system features include native email applications that can link work and personal email accounts, a calendar application to keep track of tasks, meetings and events and a contacts library to organize and search for contact information.

Mobile operating systems also include important enterprise management controls. With devices now housing both user and corporate data, it is important that these mobile OSes are secure, and data can be remotely managed and accessed if needed. Apple and Google have developed standard APIs allowing mobile device management (MDM) vendors to create platforms for securing devices that have features such as the following:

- Pin code generation and encryption.
- Provisioning Wi-Fi and certificates.
- Enabling and disabling features and capabilities.
- Remote wipe and factory reset if a device is lost or stolen.
- Safeguards to ensure end-user privacy and security for Bring Your Own Device (BYOD) programs.

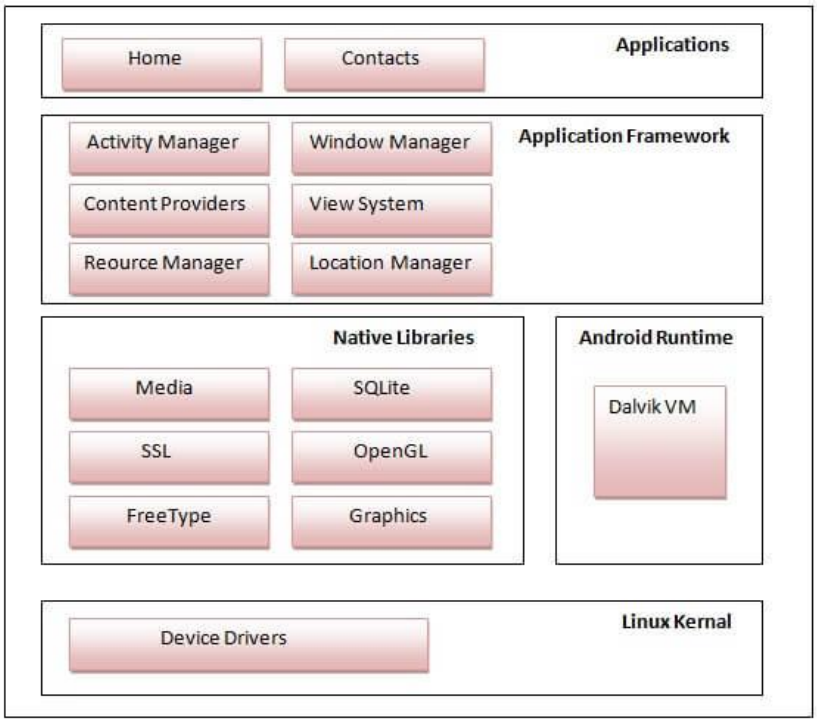
Other common features include customizable home screens and app stores, access to cloud storage solutions, advanced enterprise and corporate management security options, support for multiple accounts, multimedia capabilities such as music and video players, messaging and collaboration features, and more. Each system has unique features to help users find what best suits their needs.

I. Android OS Architecture

Android architecture or **Android software stack** contains different number of components to support any android device needs. Android software contains an open-source Linux Kernel having collection of number of C/C++ libraries which are exposed through an application framework services. Among all the components Linux Kernel provides main functionality of operating system functions to smartphones and Dalvik Virtual Machine (DVM) provide platform for running an android application. The main components of android architecture are following:-

1. Linux kernel
2. Native libraries (middleware),
3. Android Runtime
4. Application Framework
5. Applications

Let's see the android architecture first.



(1) Linux kernel: It is the heart of android architecture that exists at the root of android architecture. **Linux kernel** is responsible for device drivers, power management, memory management, device management and resource access.

(2) Native Libraries: On the top of linux kernel, their are **Native libraries** such as WebKit, OpenGL, FreeType, SQLite, Media, C runtime library (libc) etc.

The WebKit library is responsible for browser support, SQLite is for database, FreeType for font support, Media for playing and recording audio and video formats.

(3) Android Runtime: In android runtime, there are core libraries and DVM (Dalvik Virtual Machine) which is responsible to run android application. DVM is like JVM but it is optimized for mobile devices. It consumes less memory and provides fast performance.

(4) Android Framework: On the top of Native libraries and android runtime, there is android framework. Android framework includes **Android API's** such as UI (User Interface), telephony, resources, locations, Content Providers (data) and package managers. It provides a lot of classes and interfaces for android application development.

(5) Applications: On the top of android framework, there are applications. All applications such as home, contact, settings, games, browsers are using android framework that uses android runtime and libraries. Android runtime and native libraries are using linux kernal.

II iOS Architecture

What is iOS architecture?

Architecture of iOS or Apple's iOS mobile operating system runs the systems for iPhone, iPad, and iPod Touch. The title was updated with the launch of the pads originally known as the iPhone OS. This uses a multi-touch interface where the app is controlled through basic motions, such as flicking your fingers around the display to switch to the next section or rubbing your finger to focus in. The Apple App Store, the most famous app store on any smartphone, has moreover two million iPhone apps available online.



Figure iOS Operating system

The architecture of iOS.

Apple defines the collection of mechanisms and technologies currently being introduced as a sequence of layers inside the operating system. Each one of these layers comprises multiple frameworks to be used and implemented through your application areas: There are four layers in the architecture of IOS namely Core OS, Core Services, Media Layer and Cocoa Touch.

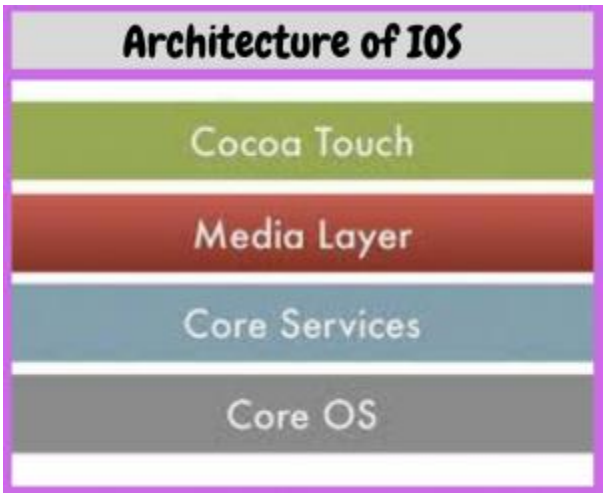


Figure . Architecture of IOS

(i) The core layer of iOS: It is the hierarchical structure's lower part and is liable for the operating system based on which the various layers are built on. This critical layer is responsible for memory management (assigning and removing memory until the program is complete), file system management, communication management, as well as other operating system activities. It specifically communicates with the hardware, too. Most of the features that the three higher tier layers have are based on the Core OS layer as well as its low-grade features. The Main OS layer includes a range of frames that can be used directly by your programs, such as the Accelerate and Protection frameworks.

For apparent security purposes, the Core OS layer also embodies the kernel framework and low-grade UNIX interfaces with which your application has no exposure. However, several low-level attributes, like BSD connectors, POSIX posts, and DNS service providers, can be powered easily via the C-based library system catalogue.

(ii) The core service layer in iOS: The core services layer is responsible for handling the basic device services that are used by native iOS users. In some of its features, the Cocoa Touch layer is based on the Core Services layer. There is also a range of important features in the Core Services tier, like block artifacts, Grand Central Control, In-App Purchase, and iCloud Memory. Automatic Reference Counting (ARC) is among the most important additions to the core services network. ARC is a preprocessor-level feature, launched with the release of iOS 5 in 2011. ARC optimizes the memory management within Objective-C. A further important framework for iOS and OS X apps is the Base Framework or Base for short. We'll discuss this concept in more depth in the next paragraph.

(iii) **Media layer in iOS: The media layer** allows the image on your iOS device to be clearer and the music you 're listening to sound transparent. All audio, video, and graphics components are included in this layer which helps in providing a dynamic immersive experience for the mobile user. The Media layer manages the graphics, audio, and video. There is a range of primary innovations in this layer, like OpenGL ES, Core Graphics, AV Foundation, OpenAL, and Core Media. The Media layer includes a large number of frames such as the Assets Libraries system for accessing the images and videos stored on the computer, the Core Image system for shutter photo manipulation, and the Central Graphics system for 2D representation.

(iv) **Cocoa Touch Layer in iOS:** The Cocoa Touch layer is therefore very essential for just how we communicate with our iOS devices. This completely covered how our applications appear when we try to communicate with them, and also how they react. For instance, if you appreciate the notification bar on your smartphone, you can thank the Cocoa Touch layer for that feature. The Cocoa Touch layer also contains a range of high-level iOS framework features, such as the opportunity to multi-task among apps or use the touchscreen of the computer to execute commands. The Cocoa Touch layer is the topmost part of the iOS structure. This includes some of the main frameworks on which native iOS apps depend, the most popular of which is the UIKit system.

The Cocoa Touch layer describes the core application architecture and includes a range of critical features, like multi-tasking and touch-based feedback. The UIKit framework is customized to the iOS platform or UIKit in short. OS X system, Application Manager application, or AppKit provides an analogous application. UIKit offers the visual, event guided iOS framework. Apple also looks after other core elements of the iOS app, such as multitasking, push alerts, and availability. The Cocoa Touch layer offers a wide range of high-level features for users, like auto design, publishing, gesture object detection, and document management.